

HEH

Synthèse Linux

Section Informatique – Orientation réseaux et télécommunications

BRUYERE Maximilien
Examen janvier 2024

Je ne suis en aucun cas responsable de vos échecs 😊

Table des matières

Chapitre 1. Introduction GNU/Linux.....	6
1. Qu'est-ce qu'un logiciel libre ?	6
2. Une licence = libre ?	6
3. Que veut dire Open Source ?	6
4. OS GNU/Linux	6
5. Distributions Linux	7
6. Caractéristiques techniques.....	7
7. Mode graphique vs texte	7
8. Connexion au système	8
9. Qu'est-ce qu'un shell, une console et un terminal ?	8
Chapitre 2. L'administrateur	9
1. Rôles de l'administrateur	9
2. Méthodologies de l'administrateur	9
3. Outils de l'administrateur	9
Chapitre 3. Gestion des fichiers.....	10
1. C'est quoi un fichier ?	10
2. Arborescence des fichiers	10
3. Répertoire personnel et répertoire courant	11
4. Les extensions de fichier	12
5. Les liens	12
6. Les droits d'accès basiques [fichier].....	13
7. Les droits d'accès étendus	13
8. Commandes.....	14
Chapitre 4. Gestions des utilisateurs et des groupes	15
1. Notions d'utilisateurs et de groupes	15
2. UID et GID	15
3. Utilisateurs et sécurité	15
4. Mécanismes d'authentification des utilisateurs	16
5. Les mots de passe	16
6. Commandes.....	17
Chapitre 5. Gestion des systèmes de fichiers	17
1. Qu'est-ce qu'un système de fichiers ?	17
2. Structure d'un disque dur	18
3. Structure du système de fichiers ext2.....	18
Le bloc d'amorçage.....	19
Le super bloc.....	19
Liste de description des groupes de blocs	19

Les cartes de blocs et d'inodes	19
La table des inodes	19
Les blocs de données	20
4. Gestion des systèmes de fichiers	20
5. Types de systèmes de fichiers	21
Système de fichiers journalisé	21
Exemples de systèmes de fichiers	22
6. Procédures pour monter convenablement un système de fichiers	22
Montage sans utiliser /etc/fstab	22
Montage avec /etc/fstab	23
7. Commandes	23
Chapitre 6. Archivages, sauvegardes et compressions	24
1. Archivage et compression	24
2. Sauvegardes	24
3. Plan de sauvegarde	25
4. Types de sauvegardes	26
5. Types de stockages	26
DAS (Direct Attached Storage)	26
NAS (Network Attached Storage)	27
SAN (Storage Area Network)	27
Comparaison SAN – NAS	28
6. Les types de supports de stockage	29
Bande magnétique	29
Disque dur	29
SSD (Solid State Drive)	30
Les supports optiques	30
Disquette	30
7. Commandes	30
Chapitre 7. Installer des programmes	31
1. Package vs dépôt compilé	31
Chapitre 8. Gestion des processus	31
1. Programme vs processus	31
2. Gestion des processus	31
Numéro du processus (Process Identification ou PID)	32
Processus parent	32
Numéro d'utilisateur et de groupe (UID et GID)	32
Durée de traitement et priorité	32
3. Exécution d'un processus	33

4.	Héritage	33
5.	Les signaux.....	33
6.	Traitement en tâche de fond	34
7.	Priorité d'un processus	34
8.	Commandes.....	35
Chapitre 9. Gestion des ressources – partie 1		35
1.	À quoi servent la commande crontab et le service crond ?.....	35
2.	Qu'est-ce que Anacron ?	36
3.	La commande at.....	36
4.	Quotas d'espace disque.....	36
	C'est quoi ?	36
	Activation des quotas.....	37
5.	Commandes.....	38
Chapitre 10. Gestion des ressources – partie 2		39
1.	Pourquoi surveiller les ressources ?	39
2.	Quelles sont les ressources à surveiller ?	39
3.	Surveillance du CPU	39
	Que faire en cas de niveau élevé d'utilisation de temps CPU ?	39
	Que faire lorsque des conflits de ressources CPU sont la source d'un goulot d'étranglement ?	39
4.	Surveillance de la mémoire	40
	Gestion de l'espace de pagination	40
	Facteurs influençant les performances des entrées-sorties sur disque	40
5.	Comment résoudre un problème de performance du système ?	41
	Poser le problème.....	41
	Déterminer la cause du problème	41
	Formuler les objectifs à atteindre.....	41
	Concevoir les modifications adéquates.....	41
	Surveiller.....	41
	Recommencer.....	41
Chapitre 11. Le noyau Linux		42
1.	Informations générales concernant les noyaux	42
	Rôle d'un OS (Operating System)	42
	Le noyau linux.....	42
	Les types de noyau	42
Chapitre 12. LVM (Logical Volume Manager)		43
1.	Qu'est-ce que c'est ?.....	43
2.	Les différents concepts	43
	Physical Volume (PV)	43

Volume Group (VG)	43
Logical Volume (LV)	44
Physical Extent (PE)	44
Logical Extent (LE)	44
Chapitre 13. Démarrage et arrêt d'un système Linux et gestion des services	44
1. Procédure de démarrage	44
2. GNU GRUB (Grand Unified Bootloader)	45
3. Init vs Systemd	45
4. Les niveaux d'exécutions (runlevels)	46
5. Les grands principes de systemd	46
6. Notion d'unité	46
7. Les fichiers associés à l'utilisateur	46
8. Fichiers de l'utilisateur	47
9. Journalisation du démarrage	48
10. Commandes	48
Chapitre 14. Mise en réseau sous Linux	49
1. Les fichiers de configuration	49
Résolution des noms	49
Paramètres de la machine	49
Base de données	49
2. NFS (Network File System)	49
Bref historique du NFS	49
Procédures pour mettre en place un serveur NFS (client & serveur)	50
3. Principe RPC (Remote Procedure Call)	53
Fonctionnement du RPC	54
Avantages et inconvénients du RPC	54
4. Commandes	55
Chapitre 15. Introduction au shell BASH et scripting shell	56
1. Qu'est-ce qu'un shell ?	56
2. BASH (Bourne Again Shell)	56
Les redirections (>)	56
Les pipes ()	56
Les commandes multiples	57
Les variables d'environnement	57
Les structures de contrôles	58
Chapitre 16. Administration de réseaux	60
1. SNMP (Simple Network Management Protocol)	60
Principe de fonctionnement de SNMP :	60

Chapitre 17. Introduction à la sécurité	62
1. Les types de menaces	62
2. Les sinistres	64
3. Principales sources de menaces.....	65
4. DRP (Disaster Recovery Plan).....	65
5. Que devons-nous sécuriser ?.....	65
6. Critères de sécurisation	66
7. Comment sécuriser ?	66
Chapitre 19. Les Firewalls	66
1. Principe général d'un pare-feu	66
Le filtrage	66
Types de pare-feu	67
Blocage des attaques par FW	68
2. Les proxys	68
Principes généraux d'un proxy.....	68
Protection par proxy	69
Avantages d'un proxy	69
Inconvénients d'un proxy.....	69
Reverse Proxy	69
3. Procédure pour désactiver firewalld et activer iptables	70
4. Tables, chaînes, règles, politiques et cibles	70
Les tables.....	70
Les chaînes	70
Les règles	71
Les politiques	71
Les cibles	71
5. Procédure pour mettre en place un firewall (iptables).....	72
6. Commandes.....	73

Chapitre 1. Introduction GNU/Linux

1. Qu'est-ce qu'un logiciel libre ?

C'est un logiciel fourni avec les autorisations d'utiliser le logiciel, de copier le logiciel, d'étudier le logiciel, de modifier le logiciel donc avoir un accès au code source et de distribuer le logiciel (forme originale – modifiée).

Remarque :

Les logiciels libres ne sont pas toujours gratuits.

2. Une licence = libre ?

Il existe une multitude de licences (ex : GPL – General Public License). La licence GPL assure que le logiciel est libre et restera libre. L'auteur du logiciel restera propriétaire du code mais n'impose aucune restriction à l'utilisateur quant aux modifications qu'il pourrait y apporter. Les utilisateurs peuvent modifier leur code et le distribuer à condition que ce code fasse parti de la licence GPL. Cette licence protège les droits des développeurs.

3. Que veut dire Open Source ?

C'est un logiciel qui répond à une série de critères dont une qui est la disponibilité du code pour pouvoir pour effectuer une étude de code.

4. OS GNU/Linux

Linux est le noyau d'un système d'exploitation (UNIX) entièrement libre et ouvert. Le système d'exploitation est une couche d'instructions logicielles ce qui permet de faire des liens entre le hardware et le software. Le cœur d'un système d'exploitation fourni des fonctions de base telles que le chargement et l'exécution d'un programme, la gestion des entrées et des sorties, l'interface graphique.

Il y a des fonctions supplémentaires :

- Une structure pour stocker les informations sur le disque dur (File System).
- Le support du réseau (TCP/IP, Firewall, ...).
- Des drivers pour le matériel.

Remarque :

Tout le monde peut utiliser Linux, allant d'un simple utilisateur lambda au développeur d'application.

5. Distributions Linux

Une distribution Linux est constituée d'un système d'exploitation à base d'un noyau Linux et d'un ensemble d'applications (elles peuvent être libres mais aussi propriétaires. Il n'y a pas de distribution standard.

Par exemple : RedHat, Fedora, Ubuntu, Debian, AlmaLinux, ...

Les types de distributions Linux :

- Minimalistes : très petits en taille (parfois seulement des Mo suffisent à l'installer).
- Permutation d'un ordinateur en Media Center (GeeXboX).
- Dédiés à la création multimédia (AGNULA/Demudi).
- Destinés à « transformer » un ordinateur en firewall (IPCop).
- Dits « LiveCD » - CD contenant le système d'exploitation sans installation préalable.

6. Caractéristiques techniques

Il y a le système de base qui comprend :

- Une architecture 32 bits et 64 bits disponibles.
- Un système multi-tâches : faire tourner plusieurs programmes simultanément.
- Des systèmes multi-utilisateurs : plusieurs utilisateurs connectés en même temps.
- Un système multi-plateformes : peut fonctionner sous différents CPU (Inter, AMD, Sun SPARC, ...).
- Une mémoire protégée : un programme ne peut pas compromettre à lui seul le fonctionnement de l'ensemble du système.
- Une gestion de volumes logiques : création de volumes (partitions virtuelles).
- Un support RAID : distribution d'une information sur plusieurs disques, permet d'augmenter la vitesse et la redondance des informations.
- Une reconfiguration dynamique : on doit, presque jamais redémarrer un serveur sous Linux car il y a la possibilité de redémarrer uniquement les services en question.
- Un système de fichiers (Files System) : prend en charge, entre autres, ext2fs, ext3fs, ext4, xfs, vfat, ...

7. Mode graphique vs texte

Le mode texte (mode console) est un mode d'affichage dans lequel les informations apparaissent à l'écran uniquement sous forme de caractères. On utilise que le clavier et est universellement disponible sur toutes les cartes graphiques. Il n'est pas gourmand en termes de performances, est préférable sur une machine serveur et est utilisé dans le mode « rescue » - dépannage.

Le mode graphique n'est pas recommandé sur une machine serveur. Il peut y avoir des failles de sécurité assez importantes, il utilise des interfaces ergonomiques (front-end), il offre beaucoup plus de possibilités que le mode texte, il y a un confort visuel assez important et est idéal pour la convivialité.

8. Connexion au système

Grâce à une procédure d'authentification avec un nom d'utilisateur et un mot de passe, vous aurez un accès à Linux. Cette procédure se nomme « procédure de login ». Le compte « root » est le super utilisateur (celui qui possède tous les droits), il est configuré dès l'installation.

Le prompt :

- Le caractère \$ indique que nous sommes connectés avec un utilisateur lambda.
- Le caractère # indique que nous sommes connectés avec le compte « root ».

```
[user2@localhost root]$
```

```
[root@localhost ~]#
```

Remarque :

L'utilisateur lambda a le contrôle total sur son dossier courant contrairement à l'utilisateur « root » qui a le contrôle total sur tous les dossiers.

9. Qu'est-ce qu'un shell, une console et un terminal ?

C'est un logiciel fournissant une interface permettant à un utilisateur d'interagir avec le système. Il y a deux types de shell :

- Les shells en mode « texte » appelé CLI (Command Line Interface). Ce sont des interpréteurs de commandes. Le shell CLI intégré de base dans Linux s'appelle le BASH (Bourne Again Shell).
- Les shells graphiques appelés GUI (Graphical User Interface) qui fournissent une interface graphique plus intuitive pour un utilisateur lambda. Cette fonctionnalité est assurée par des logiciels « gestionnaire de bureau » ou « environnement de bureau » comme GNOME ou KDE.

Le terminal est constitué des mêmes éléments écran-clavier-souris mais accède à l'ordinateur par l'intermédiaire d'un réseau. Exemple : Lorsque vous vous connectez, à partir d'un ordinateur local, à une machine virtuelle la console que vous utilisez est considérée comme un terminal.

Une console est constituée d'un ensemble écran-clavier-souris directement branchés à l'ordinateur. On utilise une console pour saisir et exécuter des commandes, et une ou plusieurs consoles pour montrer les documentations détaillant l'utilisation de ces commandes. On utilise une console pour saisir et exécuter des commandes d'administration (ex : useradd [nom_d_utilisateur]) et une autre pour vérifier que les modifications ont bien été effectuées (ex : se connecter au système avec l'utilisateur créé dans la première console).



Chapitre 2. L'administrateur

1. Rôles de l'administrateur

L'administrateur d'une entreprise doit s'occuper de nombreuses choses dont :

- La gestion des besoins, du budget et des priorités.
- La gestion des ordinateurs et des périphériques.
- La gestion des performances des systèmes.
- La gestion des utilisateurs.
- La gestion des fichiers et des disques.
- La gestion des services.
- La gestion des problèmes.
- La gestion des sauvegardes et du stockage des données.
- La gestion du réseau.
- La gestion de la sécurité.

2. Méthodologies de l'administrateur

L'administrateur doit avoir une méthodologie assez minutieuse pour être efficace. Cette méthodologie regroupe :

- Documenter ses actions.
- Sauvegarder (faire des plans de sauvegarde).
- Automatiser (pour éviter de faire des choses redondantes).
- Agir de manière réversible (pouvoir revenir en arrière en cas de problème).
- Être proactif (anticipation des problèmes).
- Savoir communiquer.
- Avoir une bonne connaissance du marché.
- Connaître ses limites.

3. Outils de l'administrateur

L'administrateur utilise généralement trois outils principaux :

- Les commandes.
- Les fichiers de configuration.
- Les scripts.

Chapitre 3. Gestion des fichiers

1. C'est quoi un fichier ?

Un fichier est une suite de bits l'un à la suite de l'autre. Il permet de stocker toutes sortes d'informations (utilisateurs, documents, ...). Dans Linux, tout est fichier. Il permet la simplification de la manipulation du système ainsi que de la programmation.

Remarque :

- Les répertoires sont en **bleu foncé**.
- Les fichiers exécutables sont en **vert**.
- Les fichiers sont en **gris**.
- Les liens symboliques (on y reviendra) sont en **cyan**.

2. Arborescence des fichiers

Cette arborescence permet d'organiser les fichiers selon leur importance. Elle commence à partir de la racine « / ». On utilise une norme pour une compatibilité entre les différentes distributions Linux, celle-ci s'appelle : FHS (File Hierarchy Standard).

/	la racine, elle contient les répertoires principaux
/bin	Contient les exécutables essentiels au système, employés par tous les utilisateurs.
/boot	Contient les fichiers de chargement du noyau, dont le chargeur d'amorce.
/dev	Contient les points d'entrée des périphériques.
/etc	Contient les fichiers de configuration nécessaires à l'administration du système (fichiers <i>passwd</i> , <i>group</i> , <i>inittab</i> , <i>ld.so.conf</i> , <i>lilo.conf</i> , ...).
/etc/X11	Contient les fichiers spécifiques à la configuration de X (contient XF86Config par exemple)
/home	Contient les répertoires personnels des utilisateurs. Dans la mesure où les répertoires situés sous <i>/home</i> sont destinés à accueillir les fichiers des utilisateurs du système, il est conseillé de dédier une partition spécifique au répertoire <i>/home</i> afin de limiter les dégâts en cas de saturation de l'espace disque.
/lib	Contient les bibliothèques standards partagées entre les différentes applications du système.
/mnt	Permet d'accueillir les points de montage des partitions temporaires (cd-rom, disquette, ...) ² .
/proc	Regroupe un ensemble de fichiers virtuels permettant d'obtenir des informations sur le système ou les processus en cours d'exécution.
/root	Répertoire personnel de l'administrateur root. Le répertoire personnel de l'administrateur est situé à part des autres répertoires personnels car il se trouve sur la partition racine, afin de pouvoir être chargé au démarrage, avant le montage de la partition <i>/home</i> .
/sbin	Contient les exécutables essentiels du système (par exemple la commande <i>useradd</i>).
/tmp	Contient les fichiers temporaires
/usr	Hiéarchie secondaire
/usr/X11R6	Ce répertoire est réservé au système X version 11 release 6
/usr/X386	Utilisé avant par X version 5, c'est un lien symbolique vers <i>/usr/X11R6</i>
/usr/bin	Contient la majorité des fichiers binaires et commandes utilisateurs
/usr/include	Contient les fichiers d'en-tête pour les programmes C et C++
/usr/lib	Contient la plupart des bibliothèques partagées du système
/usr/local	Contient les données relatives aux programmes installés sur la machine locale par le root
/usr/sbin	Contient les fichiers binaires non essentiels au système et réservés à l'administrateur système
/usr/share	Réservé aux données non dépendantes de l'architecture
/usr/src	Contient des fichiers de code source
/var	Contient des données versatiles telles que les fichiers de bases de données, les fichiers journaux (logs), les fichiers du spouleur d'impression ou bien les mails en attente.

3. Répertoire personnel et répertoire courant

Le répertoire personnel (home directory) d'un utilisateur contient toutes les données d'un utilisateur et les paramètres personnels du compte utilisateur. Cet utilisateur est maître dans ce dossier c'est-à-dire qu'il peut créer des fichiers et/ou des dossiers, en supprimer et les modifier. Ce dossier se trouve dans */home* sauf pour le répertoire personnel du super utilisateur root qui se trouve, quant à lui, dans */root*. Le répertoire courant est le dossier dans lequel l'utilisateur se trouve.

4. Les extensions de fichier

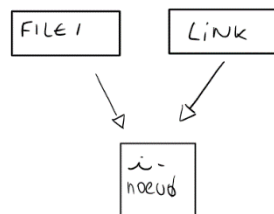
Le système Linux ne gère pas d'extension, néanmoins certains programmes nécessitent de respecter la convention de nommage liée à l'extension du fichier.

Extension	Fichier
.c	Fichier source du langage C
.bz2	Fichier compressé par la commande bzip2
.h	Fichier d'en-tête du langage C
.gz	Fichier compressé au format ZIP par la commande gzip
.png	Format ouvert d'images numériques
.rpm	Paquetage de la distribution RedHat
.sh	Script shell
.tar	Archive de type TAR (Tape ARchive)
.tgz	Fichier TAR compressé par gzip
.txt	Fichier texte

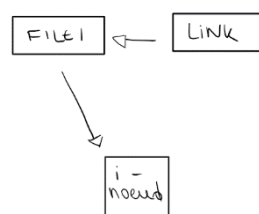
5. Les liens

Il y a deux types de liens que l'on peut avoir dans Linux :

- Les liens durs redirigent vers le même i-nœud. Les modifications faites sur un fichier se répercuteront sur tous les autres portant le même i-nœud. Si l'un des fichiers portant un i-nœud x est supprimé, aucun des autres portant le même i-nœud ne sera supprimé aussi.



- Les liens symboliques n'utilisent pas les i-nœud, mais établissent un lien vers le fichier en question, c'est pour cela que si l'on supprime le fichier original, le lien sera dit cassé car il n'y aura plus de lien possible vers l'i-nœud. Il n'est pas possible de donner plusieurs noms à des répertoires. Il n'est pas non plus possible de créer un lien dur entre deux fichiers situés sur des systèmes de fichiers différents.



6. Les droits d'accès basiques [fichier]

Il y a trois catégories d'utilisateurs :

- Le propriétaire du fichier.
- Les membres du groupe auquel est associé le fichier.
- Les autres utilisateurs (ne faisant pas partie de la première et de la deuxième catégorie).

Les droits d'accès des fichiers sont les suivants :

- R : read = lire dans le fichier.
- W : write = écrire dans le fichier.
- X : execute = exécuter un fichier.

Les droits d'accès des dossiers sont les suivants :

- R : read = consulter le contenu d'un répertoire.
- W : write = ajouter et/ou supprimer des fichiers dans le répertoire.
- X : execute = se déplacer dans ce répertoire.

7. Les droits d'accès étendus

Les droits d'endossement permettent à un utilisateur quelconque d'obtenir les droits du propriétaire du fichier lors de l'exécution du fichier. Ces droits ne sont disponibles que pour les répertoires et les exécutables. Lorsque le droit d'endossement s'applique au groupe associé à un répertoire, les fichiers qui sont créés dans ce répertoire, sont automatiquement associés au groupe du répertoire et non au groupe de l'utilisateur qui les crée. En gros, les fichiers « héritent » du groupe du répertoire.

Quand on ajoute les droits d'endossement à l'utilisateur, on parlera de SUID. Le GUID, c'est quand on ajoute les droits d'endossement au groupe associé à un répertoire.

Le sticky bit ou bit « t » permet, dans le cas d'un fichier exécutable, de le laisser en mémoire même après la fin de son exécution. Cela lui permettra d'être relancé plus rapidement en cas d'utilisation fréquente. Dans le cas d'un répertoire, il signifie qu'un fichier situé dans ce répertoire ne pourra être modifié que par son propriétaire. Par exemple, tout utilisateur peut copier des fichiers dans le répertoire /tmp (droits rwx pour tout le monde). Pour que ces fichiers ne puissent pas être effacés par un autre utilisateur que celui qui les a créés, il faut positionner ce sticky bit.

Exemple : `chmod 1777 /tmp`

Attention, l'utilisation du droit d'endossement avec un fichier exécutable présente des risques du point de vue de la sécurité car elle « donne » le droit d'accès à une commande et pas à une catégorie d'utilisateurs. Cette technique ne doit donc pas être généralisée, surtout quand elle permet l'exécution en tant qu'utilisateur root.

8. Commandes

- a. Lister le contenu d'un répertoire : **ls**
- b. Se déplacer dans un répertoire : **cd**
- c. Création d'un fichier : **touch**
- d. Suppression d'un fichier : **rm**
- e. Copier un fichier ou un répertoire : **cp**
- f. Comparer deux fichiers : **cmp**
- g. Renommer et/ou déplacer un fichier ou un répertoire : **mv**
- h. Créer un lien vers un fichier et/ou un dossier : **ln**
- i. Rechercher le manuel d'une commande : **man**
- j. Lister les fichiers ouverts : **lsuf**
- k. Indiquer le type de fichier : **file**
- l. Chercher un fichier dans l'arborescence : **find**
- m. Localiser une commande : **which**
- n. Renvoyer le statut d'un fichier : **stat**
- o. Définir les autorisations et permissions lors de la création d'un fichier et/ou répertoire :
umask*
- p. Afficher le contenu d'un fichier : **cat**
- q. Rechercher l'occurrence dans un fichier : **grep**
- r. Afficher l'entête d'un fichier : **head**
- s. Afficher la fin d'un fichier : **tail**
- t. Afficher le nombre de lignes d'un fichier texte : **wc**
- u. Afficher ou changer la date d'un système : **date**
- v. Ordonner l'arrêt du système : **halt**
- w. Afficher un texte dans le terminal : **echo**
- x. Visualiser l'historique des commandes passées : **history**
- y. Redémarrer/rebooter le pc : **reboot**
- z. Localiser un binaire : **whereis**
- aa. Arrêter le système : **shutdown**
- bb. Visualiser le calendrier du mois actuel : **cal**
- cc. Effacer l'interface du terminal : **clear**
- dd. Localiser les fichiers par leur nom : **locate**
- ee. Visualiser le répertoire courant : **pwd**
- ff. Visualiser l'utilisateur actuel : **whoami**

- gg. Ajouter des répertoires : **mkdir**
- hh. Supprimer des répertoires (vides) : **rmdir**
- ii. Écrire du texte dans le terminal : **echo**

Chapitre 4. Gestions des utilisateurs et des groupes

1. Notions d'utilisateurs et de groupes

Un compte utilisateur peut être utilisé par des utilisateurs physiques qui se connectent au système via un login et un mot de passe. On parle alors de comptes « physiques ». Il existe également des comptes dits « systèmes » ou « logiques », utilisés uniquement par des applications spécifiques. En parcourant le fichier `/etc/passwd`, vous constaterez que le système, dès son installation, a créé une série de « comptes systèmes » et un seul compte physique : `root`.

Un groupe est un regroupement d'utilisateurs qui partagent les mêmes permissions sur un ensemble de fichiers (exécutables et non) et répertoires. Les utilisateurs membres d'un groupe donné disposent de toutes les permissions des fichiers et répertoires associés à ce groupe.

Chaque utilisateur doit faire partie au moins d'un groupe : son groupe primaire. Un compte utilisateur peut être membre de plusieurs autres groupes, ceux-ci sont appelés ses groupes secondaires.

Dans les distributions de type RedHat, Linux utilise un système de groupes propres à l'utilisateur appelé UPG (User Private Group) dont le but est de faciliter la gestion des groupes d'utilisateurs. Un UPG est créé chaque fois qu'un nouvel utilisateur est ajouté au système. Par défaut, les UPG portent le même nom que le compte utilisateur pour lequel ils ont été créés et seul cet utilisateur est membres de l'UPG.

2. UID et GID

Chaque utilisateur se voit attribuer un numéro d'identification unique : User Identifier (UID). Il en est de même pour les groupes, qui sont identifiés par leur Group-Identifier (GID).

3. Utilisateurs et sécurité

Utilisation du compte `root` : tâches administratives.

Utilisation du compte personnel : tâches quotidiennes.

Pourquoi ? Tout simplement pour la sécurité car si un virus prend le contrôle de la machine alors que vous êtes en `root`, il aura probablement accès à toutes les données s'y trouvant alors que si vous étiez sur votre compte personnel, l'impact aurait été limité au compte utilisateur.

Les tâches administratives doivent être vérifiées deux fois avant de les faire.

4. Mécanismes d'authentification des utilisateurs

Les différents utilisateurs (locaux ou distants) d'un système (client ou serveur) n'ont pas tous les mêmes droits d'accès sur ces systèmes. Pour la sécurité, les accès aux systèmes et aux services reposent sur deux opérations essentielles :

- L'identification.
- L'authentification.

L'identification consiste à préciser qui est l'utilisateur (compte utilisateur) afin de déterminer quels sont ses privilèges. L'authentification consiste à prouver que l'utilisateur est bien celui qu'il prétend être. Pour mener à bien l'authentification, les informations relatives aux utilisateurs doivent être stockées quelque part. Avant ces informations étaient stockées dans le fichier : `/etc/passwd`, cependant les données à l'intérieur étaient en mode « lecture seule » pour tout le monde. À présent, elles sont stockées dans le fichier `/etc/shadow` qui n'est lisible que par root.

Cette configuration n'est pratique que pour un usage local mais n'est pas pratique pour la gestion de profils itinérants (serveur). Dans ce genre de configuration, nous allons préférer utiliser un service réseau pour récupérer les informations concernant les utilisateurs à partir d'un serveur d'authentification centralisé (LDAP par exemple).

5. Les mots de passe

Les mots de passe constituent un point faible dans une stratégie de sécurité. D'une part, ils sont souvent négligés par les utilisateurs : noté sur des post-it, confié à un collègue, etc. D'autres part, ils peuvent être crackés parfois très facilement.

Les mots de passe ne doivent pas être stockés en clair sur un système, mais sous une forme chiffrée. Linux utilise le système shadow password (mots de passe masqués). Ce système permet de placer les mots de passe chiffrés dans `/etc/shadow`, lisible seulement par root, et autorise la gestion d'informations sur l'expiration de mots de passe.

Pour améliorer la sécurité sur les systèmes critiques, une bonne solution consiste à utiliser des systèmes d'authentification plus sûrs, voir plusieurs en série (carte magnétique, carte à puce, ...) mais il faut aussi garder un compromis entre la facilité d'usage / gestion et l'efficacité de la sécurité.

Malheureusement, certains utilisateurs et/ou administrateurs utilisent des séquences simples comme mots de passe (ex : 123456, azerty, Test123*, ...) ou encore des significations simples à retenir (ex : le nom du chien, date de mariage, ...). Même si parfois, ils remplacent une lettre par un symbole (ex : a = @, s = \$, ...), ces mots de passe peuvent être assez simple à trouver avec une technique de craquage (ex : attaque par dictionnaire, ...) souvent utilisé avec un logiciel. Cela permet de les trouver en seulement quelques secondes.

Pour les mots de passe un peu plus complexes, mais basés sur des informations personnelles à l'utilisateur. En effet, reprenons l'exemple du nom de chien / mariage, autour d'une tasse de café, ces informations peuvent être facilement trouvées. Cette technique porte le nom de social engineering (extirpation des informations à des personnes sans qu'elles ne s'en rendent compte).

6. Commandes

- a. Ajouter un utilisateur : **useradd**
- b. Afficher les dates d'expirations d'un utilisateur Linux : **chage**
- c. Changer les droits sur un fichier : **chmod**
- d. Changer le propriétaire d'un fichier : **chown**
- e. Changer le propriétaire d'un groupe : **chgrp**
- f. Supprimer un utilisateur : **userdel**
- g. Renvoyer les informations UID – GID d'un utilisateur : **id**
- h. Démarrer une session sur le système : **login**
- i. Changer le mot de passe d'un utilisateur Linux : **passwd**
- j. Vérifier l'intégrité des mots de passe : **pwck**
- k. Modifier un compte utilisateur : **usermod**
- l. Afficher la liste des utilisateurs connectés à une machine : **who**
- m. Montrer le nom d'utilisateur courant : **users**
- n. S'identifier à un autre utilisateur : **su**
- o. Modifier le nom complet et les informations associées à un utilisateur : **chfn**
- p. Verrouiller un mot de passe : **passwd -l**
- q. Visualiser les informations concernant un compte : **useradd -D**
- r. Créer un groupe : **groupadd**
- s. Supprimer un groupe : **groupdel**
- t. Modifier un groupe : **groupmod**

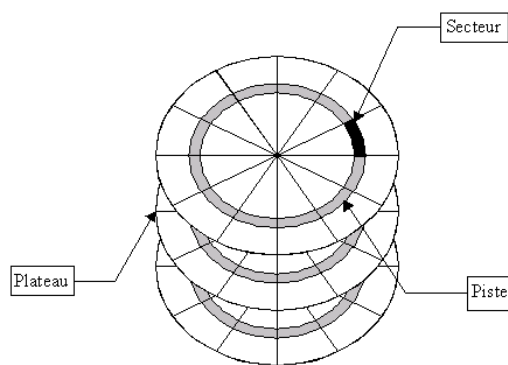
Chapitre 5. Gestion des systèmes de fichiers

1. Qu'est-ce qu'un système de fichiers ?

Le système de fichiers ou File System (FS), c'est la manière dont le système d'exploitation structure les données sur un support de stockage (disque dur, ssd, ...). Cette structure est nécessaire afin de retrouver et accéder rapidement à un fichier. Il ne faut pas confondre l'arborescence avec le système de fichiers car l'arborescence est juste une représentation logique de l'ensemble des fichiers et des répertoires présents sur le système. Les systèmes de fichiers disposent de sections distinctes du disque dur allouées pour leur utilisation. Ils sont montés automatiquement lorsque le système est démarré de sorte que l'utilisateur ne voit pas la différence entre ces systèmes de fichiers et les répertoires.

2. Structure d'un disque dur

Un disque dur est composé de pistes circulaires concentriques découpées en arcs de cercles appelés secteurs. Ceux-ci sont repérés par un couple (n°piste / n°secteur) et sont formés d'un ensemble d'octets (128, 256, 512, 1024, ... octets). Un disque dur étant un périphérique de stockage de type bloc, la modification d'un octet ou d'un bit n'est pas faisable, tous les échanges avec le disque (lecture et écriture) se font par blocs. Un bloc peut correspondre à un, deux ou plusieurs secteurs mais jamais moins qu'un secteur car le secteur est indivisible.



Les systèmes de fichiers tels que ext2 considèrent le disque dur comme une succession de blocs (ayant chacun un numéro : leur adresse physique) dont ils rangeront les adresses dans une table afin de les retrouver plus rapidement. Les fichiers, suivant leur taille, sont stockés dans un ou plusieurs blocs, de préférence, contigus. Si les blocs contenant le fichier ne sont pas contigus, le fichier sera fragmenté (réparti un peu partout dans le disque). Trop de fragmentations risquent de faire ralentir la machine, c'est pourquoi il existe des logiciels qui permettent de défragmenter les systèmes de fichiers.

3. Structure du système de fichiers ext2

Les systèmes de fichiers ext2 se compose de plusieurs éléments (rassemblés en groupes de blocs) :

Groupe de blocs 1	Bloc d'amorçage
	Superbloc
	Liste de description des groupes de blocs
	carte de blocs
	carte d'inodes
	Table des inodes
	Blocs de données

Remarque :

Chaque groupe de blocs équivaut à 8192 octets.

Le bloc d'amorçage

Le bloc d'amorçage est le premier bloc (bloc n°0) mais ne se trouve pas dans les groupes de blocs car il contient un programme qui lance et initialise le système. Sa taille est de 1024 octets en ext2.

Le super bloc

Le super bloc est crucial pour un système de fichiers car il contient des informations importantes pour son bon fonctionnement qui permettent, entre autres, de définir la structure et l'état d'un système de fichiers et permettent la gestion de toutes les informations qui y sont contenues. En voici quelques exemples :

- La taille totale du système de fichiers (en blocs et en inodes)
- Le nombre de bloc libres
- Le nombre de blocs réservés aux inodes
- La taille d'un bloc de données
- L'heure de la dernière modification effectuées
- L'heure de la dernière vérification du système de fichiers

Le système de fichiers comporte plusieurs copies du super bloc, permettant d'en trouver un intact si l'un d'eux est défectueux.

Liste de description des groupes de blocs

Derrière chaque super bloc figure une liste de description des groupes de blocs. Celle-ci permet de connaître rapidement l'adresse des groupes de blocs et inodes, ce qui facilite l'accès aux blocs de données. Elle répertorie également le nombre de blocs ou d'inodes libres.

Les cartes de blocs et d'inodes

Ces cartes répertorient l'occupation des blocs et des inodes, permettant au système de connaître rapidement les zones libres. Pour savoir si un bloc ou un inode est utilisé ou non, on lui affecte un bit (1 = utilisé, 0 = libre).

La table des inodes

Un inode ((Information Node - Nœud d'information) est un enregistrement de taille fixe qui contient « tous » les éléments décrivant un fichier mis à part le nom du fichier.

- Le type de fichier (ordinaire, répertoire, lien, ...)
- La taille du fichier
- Le propriétaire et le groupe associé au fichier
- Les permissions d'accès associés au fichier
- Les dates de création, modification et accès
- Des pointeurs vers des blocs de données (emplacement réel des données)
- Le nombre de liens durs
- ...

Lorsque l'on crée un fichier dans un répertoire, les informations sur ce fichier ne sont pas inscrites dans le répertoire mais dans un inode non affecté. Seuls le numéro de cet inode et le nom du fichier sont inscrits dans le répertoire. Le répertoire est en réalité un fichier particulier qui contient des liens, c'est à dire un couple « nom de fichier + numéro d'inode ». C'est donc le répertoire qui permet de retrouver un fichier à partir de son nom.

La table des inodes est une composante clé du système de fichiers, regroupant tous les inodes en groupes de blocs. Sa taille, déterminée lors de l'installation, limite le nombre de fichiers créables. Certains inodes sont réservés, par exemple, le premier mémorise les "bad blocs". Un compromis entre la taille de la table et le nombre de fichiers potentiels doit être trouvé. Par exemple, une table plus grande permet plus de fichiers mais peut ralentir les accès.

Remarque :

Un fichier n'est supprimé que lorsque son nombre de liens tombe à zéro.

La commande rm utilise l'appel système unlink, qui détruit un lien. Chaque suppression d'un lien réduit le nombre de liens matériel dans l'inode. L'effacement d'un fichier est rapide, car il supprime seulement le lien d'accès aux données sur le disque, sans effacer réellement les données.

Les blocs de données

Les fichiers sont stockés dans des blocs de données, repérés par des pointeurs dans l'inode. Certains pointeurs sont directs, d'autres sont indirects, voire doublement indirects. Lorsque les pointeurs directs sont épuisés, un nouveau bloc est alloué sur le disque pour stocker des pointeurs vers d'autres blocs. Le premier pointeur indirect de l'inode pointe vers ce nouveau bloc alloué.

4. Gestion des systèmes de fichiers

L'utilisateur moyen se contente de l'arborescence des fichiers, laissant la gestion des systèmes de fichiers à l'administrateur. Ce dernier peut choisir la configuration par défaut ou personnaliser celle-ci. Gérer les systèmes de fichiers présente plusieurs avantages :

- La sécurité est renforcée en utilisant plusieurs systèmes de fichiers, limitant les dommages en cas de problème.
- Les performances sont améliorées en réduisant la fragmentation des fichiers.
- La répartition de l'espace disque sur plusieurs systèmes isole les utilisateurs, offrant un meilleur contrôle de l'espace utilisé.
- La gestion des backups est simplifiée en définissant des systèmes de fichiers de taille adaptée aux unités d'archivage.

Pour accéder à un nouveau système de fichiers, celui-ci doit être monté sur un répertoire du système actif. Cette action, appelée « montage », permet d'intégrer le nouveau système de fichiers dans l'arborescence existante. Le répertoire utilisé pour le montage est appelé point de

montage, et c'est à partir de là que l'utilisateur peut accéder au nouveau système de fichiers. Cette opération est nécessaire pour rendre le système de fichiers accessible à l'utilisateur. Linux prend en charge le montage de différents types de systèmes de fichiers, qu'ils soient sur des partitions de disque dur, des périphériques amovibles (CD, disquettes, clés USB) ou des supports réseau (partages NFS, Samba, etc.). Il autorise également le montage d'un système de fichiers sur un autre, permettant ainsi un accès transparent aux données.

Pour monter des systèmes de fichiers dans l'arborescence, l'administrateur Linux peut utiliser le fichier `/etc/fstab` pour des montages automatiques au démarrage ou lors de la détection de périphériques. La commande `mount` permet également de monter manuellement avec trois paramètres : le type de système de fichiers, le fichier de périphérique et le répertoire de montage. Exemple : `mount -t iso9660 /dev/cdrom /media/cdrom`.

Les commandes `mount`/`umount` suivent les directives de `/etc/fstab`, simplifiant le montage avec des configurations préétablies. Le fichier `/etc/fstab` préconfigure les montages, évitant le montage manuel à chaque démarrage et automatisant les montages/démontages. La commande `mount` affiche tous les systèmes de fichiers intégrés.

Attention : Si le répertoire de montage n'est pas vide lors du montage, les fichiers qu'il contient ne sont pas visibles mais réapparaissent après le démontage. Linux enregistre les paramètres de montage dans `/etc/mtab`. Lorsqu'un système de fichiers n'est plus nécessaire, il peut être détaché (démonté) à l'aide de la commande `umount`. Cette commande utilise les mêmes paramètres que ceux utilisés pour le montage.

Avant de démonter un système de fichiers, l'administrateur doit s'assurer qu'aucun fichier de ce système n'est en cours d'utilisation par un utilisateur ou un processus. Si le système de fichiers est encore actif, un message d'erreur indiquant qu'il est occupé (busy) sera retourné. Dans ce cas, l'administrateur peut choisir d'attendre que les utilisateurs quittent le système de fichiers d'eux-mêmes ou de les inviter expressément à le faire.

5. Types de systèmes de fichiers

Système de fichiers journalisé

Le cache est utilisé pour accélérer les opérations d'entrée/sortie en stockant temporairement les données avant de les écrire sur le disque. En cas d'arrêt brutal, les données non écrites sont perdues, nécessitant une vérification du système de fichiers au redémarrage. Cette vérification examine inodes, blocs de données et répertoires pour détecter et corriger les incohérences. Pour une grande partition, la restauration de l'intégrité après un arrêt brutal peut prendre beaucoup de temps, rendant le système indisponible.

Exemple :

- Lorsqu'un fichier est modifié, la modification peut d'abord être stockée dans le cache avant d'être écrite sur le disque.
- En cas de coupure de courant, les données non encore écrites sur le disque peuvent être perdues.
- La vérification du système de fichiers au redémarrage examine et corrige les éventuelles incohérences causées par la perte de données non écrites.

Sur les systèmes de fichiers journalisés, les données sont mises en cache, mais la liste de ces données est enregistrée dans un fichier journal sur le disque. En cas de coupure de courant, la vérification est plus rapide car elle se limite aux données répertoriées dans le journal. Les données qui n'étaient pas en cours d'utilisation et donc pas dans le cache ne nécessitent pas de vérification. Lorsque l'on travaille avec des systèmes de fichiers journalisés, il y a une diminution des performances mais celle-ci est compensée par leurs avantages.

Exemples de systèmes de fichiers

- a. FAT (FAT12 ; FAT16,T32) : système de fichiers qui permet le montage de systèmes de fichiers MS-DOS dans l'arborescence Linux.
- b. NTFS (New Technology FileSystem) : système de fichiers de MS-Windows, plus performant et sécurisé que FAT.
- c. exFAT : évolution de la FAT destinée à la remplacer sur les supports amovibles notamment.
- d. ext2 : le standard des systèmes de fichiers sous Linux.
- e. ext3 : il s'agit d'un ext2 auquel est ajouté un système de journalisation.
- f. HFS (Hierarchical File System) : développé par Apple Inc. pour le système d'exploitation Mac OS.
- g. HFS+ : successeur de HFS.
- h. HPFS (High Performance File System) : le système de fichiers natif d'OS/2.
- i. JFS (Journaled File System) : système de fichiers journalisé disponible notamment sur les plates-formes AIX, Linux et OS/2. La version 2 (JFS2) permet notamment une allocation dynamique des inodes.
- j. ReiserFS : système de fichiers journalisé optimisé pour le traitement de très nombreux fichiers de petites tailles.
- k. NFS (Network File System) : protocole (on parle aussi de système de fichiers en réseau) développé par Sun Microsystems qui permet de partager des données principalement entre systèmes UNIX (et avec Windows depuis la version 4)
- l. CIFS (Common Internet File System) : anciennement Server Message Block (SMB), protocole réseau permettant le partage des fichiers (mais aussi des imprimantes, ...) entre différents ordinateurs d'un réseau.
- m. Système de fichiers pour CDROM : ISO9660, Joliet, Rock Ridge, CD-i, UDF
- n. /proc : système de fichiers particulier qui permet d'accéder à la mémoire du noyau.
- o. RAMFS : système de fichiers utilisé pour créer des disques en mémoire vive.
- p. ...

6. Procédures pour monter convenablement un système de fichiers

Montage sans utiliser /etc/fstab

▪ Identifier le périphérique et le point de montage

Il faut trouver le périphérique du système de fichiers que vous voulez monter. Vous pouvez utiliser la commande **lsblk** ou **fdisk -l** pour lister les périphériques. Choisissez ou créez un répertoire qui vous servira de point de montage.

- **Utiliser la commande mount**

La syntaxe générale est :

`mount -t type_de_système_de_fichiers périphériques point_de_montage`

Exemple : `mount -t ext4 /dev/sdX1 /mnt/mon_point_de_montage`

- **Vérifier le montage**

Utilisez la commande `df -h` pour voir les systèmes de fichiers montés et leurs statistiques.

- **Démonter le système de fichiers lorsque vous avez terminé**

Utilisez la commande `umount`. Exemple : `umount point_de_montage`

Montage avec /etc/fstab

- **Éditer le fichier /etc/fstab**

Ouvrez le fichier `/etc/fstab` avec un éditeur de texte en tant que super-utilisateur.

Ajoutez une ligne pour le montage automatique avec les informations nécessaires. Par

exemple : `/dev/sdX1 /mnt/mon_mount_point ext4 defaults 0 2`

- **Effectuer le montage automatique**

Utilisez la commande `mount -a` pour monter tous les systèmes de fichiers répertoriés dans `/etc/fstab`.

- **Vérifier le montage**

Utilisez la commande `df -h` pour voir les systèmes de fichiers montés et leurs statistiques.

- **Démonter le système de fichiers lorsque vous avez terminé**

Utilisez la commande `umount`. Exemple : `umount point_de_montage`

7. Commandes

a. Visualiser les PID (Process Identifier) des processus qui utilisent le système de fichiers :

`fuser`

b. Visualiser tous les fichiers actuellement ouverts : `lsdf`

c. Monter un système de fichiers : `mount`

d. Démonter un système de fichiers : `umount`

- e. Répertorier tous les blocs de stockage, y compris les partitions de disque et les lecteurs : **lsblk**
- f. Vérifier et réparer un système de fichiers Linux : **fsck**
- g. Afficher l'espace disque et les inodes libres + afficher les systèmes de fichiers montés et leurs statistiques : **df**
- h. Afficher de nombreuses informations d'un système de fichiers : **tune2fs -l**
- i. Création d'un système de fichiers en ext2 : **mke2fs**
- j. Autoriser l'administrateur à lire et/ou modifier n'importe quel bloc du système de fichiers pour le déboguer : **debugfs**
- k. Afficher les informations sur le superbloc et les groupes de blocs : **dumpe2fs**
- l. Rechercher les blocs physiques endommagés d'une partition : **badblocks**

Chapitre 6. Archivages, sauvegardes et compressions

1. Archivage et compression

La compression consiste à réduire la taille d'un ensemble d'informations numériques (fichiers, programmes, ...) de manière à leur permettre d'occuper le moins de place. Cette technique est beaucoup utilisée pour télécharger de grands fichiers sur Internet. Sur Linux, il existe plusieurs instruments de logiciel comme gzip et bzip2 ou même encore zip pour éviter les problèmes de compatibilité avec les autres systèmes d'exploitation (extension : gz, bz2, zip).

L'archivage est une technique permettant de placer plusieurs fichiers sous forme compactée ou non « à l'intérieur » d'un seul fichier (archive). La manipulation et le transfert de ces fichiers est facilitée car il suffit de manipuler le seul fichier archive. Il existe plusieurs formats d'archives, mais les plus connus sont les formats .zip et .rar. Sous Linux, c'est la commande tar qui est utilisée pour archiver (et/ou compresser)

En résumé, l'archivage concerne l'organisation et le regroupement de fichiers et de répertoires, tandis que la compression se concentre sur la réduction de la taille des données. Souvent, les deux concepts sont utilisés ensemble : on crée une archive pour regrouper des fichiers, puis on comprime cette archive pour économiser de l'espace de stockage ou faciliter les transferts. Un exemple courant est l'utilisation de formats comme ZIP ou TAR (archivage) combinés avec des algorithmes de compression tels que gzip ou bzip2.

2. Sauvegardes

L'une des tâches les plus importantes est la sauvegarde, en effet il doit mettre en place une stratégie de sauvegarde adaptée et vérifier l'intégrité des fichiers (si les fichiers sont toujours disponibles, ...). Les sauvegardes vont de la simple copie au clonage complet du système.

Linux propose plusieurs commandes de gestion de sauvegarde :

- La commande tar (Tape Archive) est utilisée pour sauvegarder des fichiers ou des arborescences.
- La commande cpio (copy in/out) est utilisée pour des transferts de données mais peut être utilisée en tant que programme de sauvegarde.
- La commande dd (disk dump) réalise des copies physiques, elle peut être utilisée pour sauvegarder des disques et des systèmes de fichiers.
- Les commandes dump et restore sont utilisées pour sauvegarder des systèmes de fichiers non-montés.

La sauvegarde consiste à créer une copie strictement identique des données et la stocker sur un support. L'archivage doit répondre, contrairement à la sauvegarde, à un besoin d'accès permanent aux données.

3. Plan de sauvegarde

Il y a de nombreuses questions à se poser pour mettre en place une sauvegarde correctement :

1. Que faut-il sauvegarder ?
2. Avec quelle fréquence ?
3. Combien de temps conservera-t-on les sauvegardes, en combien d'exemplaires ?
4. A quel endroit seront stockées les sauvegardes et l'historique des sauvegardes ?
5. Quels sont les besoins, en capacité, du support de sauvegarde ?
6. Combien de temps, au plus, doit durer la sauvegarde ?
7. Combien de temps prévoit-on pour restaurer un fichier, un système de fichiers, est-ce raisonnable ?
8. La sauvegarde doit-elle être automatique ou manuelle ?
9. Quelle est la méthode de sauvegarde la plus appropriée ?
10. Quel est le support le plus approprié ?
 - Les performances,
 - La fiabilité,
 - La pérennité,
 - La portabilité,
 - Le prix des supports,
 - Le prix des unités de lecture/écriture,
 - La compatibilité,
 - La robustesse,
 - L'encombrement (comprend le lieu de stockage).

Remarque :

Effectuer des tests de restauration est essentiel pour garantir l'efficacité d'une sauvegarde. La consultation d'archives, comme celles au format tar, répond à deux besoins :

- Vérifier l'intégrité et la correspondance du contenu de l'archive avec les attentes.
- Assurer que les noms de fichiers se réintègrent correctement dans l'arborescence lors de la restauration.

Il est important de noter qu'un fichier stocké dans un format spécifique sur un système peut être utilisé sur d'autres systèmes. Cette flexibilité est l'un des avantages des systèmes ouverts.

Exemple :

- Vérifier qu'une archive tar est complète et contient les fichiers attendus.
- S'assurer que la restauration d'une sauvegarde maintient l'intégrité des noms de fichiers dans la structure du système.

4. Types de sauvegardes

Le but d'une sauvegarde est de disposer d'une copie des données dans le cas où les données seraient perdues et/ou altérées.

Il y en a de plusieurs types :

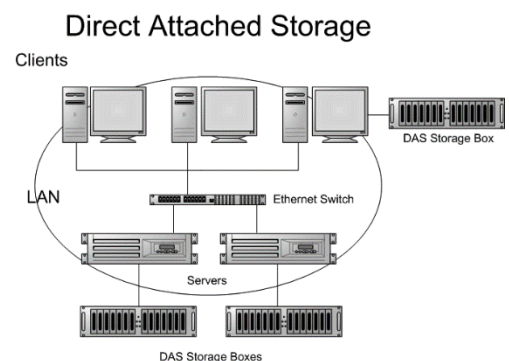
- La sauvegarde totale permet de réaliser une copie conforme des données à sauvegarder sur un support séparé. C'est très lent s'il y a beaucoup de données et si elles sont modifiées en même temps que la sauvegarde. Elle donne une image fidèle des données à un moment t.
- La sauvegarde différentielle se focalise sur les données modifiées depuis la dernière sauvegarde totale. Elle est beaucoup plus coûteuse en espace de stockage qu'une sauvegarde incrémentale mais aussi beaucoup plus fiable car il ne faut que la sauvegarde complète pour reconstituer les données sauvegardées.
- La sauvegarde incrémentale consiste à copier tous les éléments modifiés depuis la sauvegarde précédente. C'est plus performant qu'une sauvegarde totale car elle se focalise sur les fichiers modifiés avec un espace de stockage plus faible. Cependant, pour restaurer la sauvegarde complète, il lui faut toutes les sauvegardes précédentes.
- La sauvegarde à delta est comme la sauvegarde incrémentale sauf qu'elle ne se focalise pas sur les fichiers mais sur les blocs de données.
- La sauvegarde Bare metal.

5. Types de stockages

DAS (Direct Attached Storage)

Système de disque en attachement direct (opposition au NAS → attachement réseau). Le système disque n'est alors disponible directement qu'aux ordinateurs auquel il est raccordé, le plus souvent en protocole USB.

Il faut que la machine sur lequel le DAS est attaché soit allumée.



NAS (Network Attached Storage)

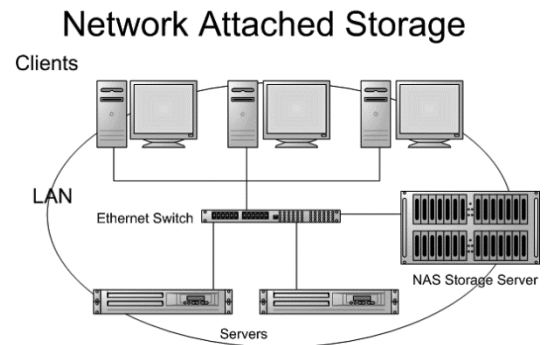
Il s'agit d'un serveur de stockage à part entière pouvant être facilement attaché au réseau de l'entreprise afin de servir de serveur de fichiers et fournir un espace de stockage tolérant aux pannes.

Il dispose de son propre OS, son propre système de fichiers et d'un ensemble de disques indépendants qui servent à héberger des données.

Il peut offrir de nombreuses fonctionnalités telles que le FTP, RAID, support de plusieurs FS (NFS, CIFS, ...).

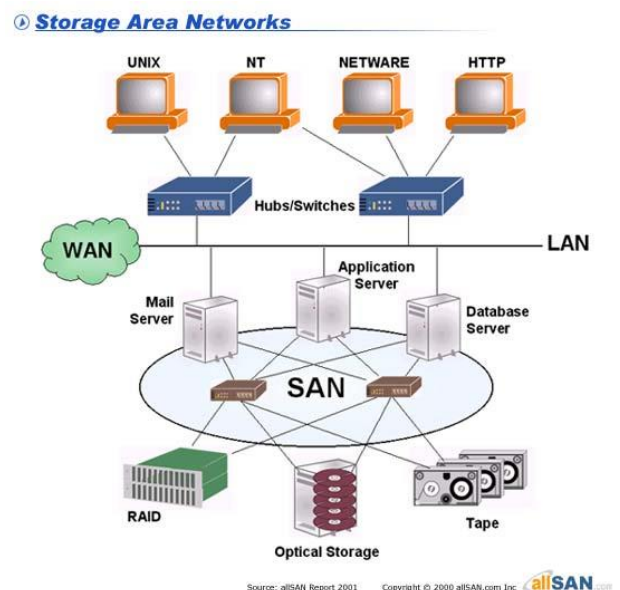
Voici les avantages du NAS :

- Il permet de faciliter la gestion des sauvegardes des données d'un réseau.
- Il y a un prix intéressant des disques de grandes capacités par rapport à l'achat de disques en grand nombre sur chaque serveur de réseau.
- Il permet à plusieurs postes clients d'accéder aux mêmes données stockées.
- Il y a une réduction du temps d'administration des postes clients en gestion d'espace disques.



SAN (Storage Area Network)

Le SAN, ou Storage Area Network, est un réseau spécialisé dédié au stockage de données. Il permet de connecter des serveurs et des dispositifs de stockage, comme des baies de disques, via une infrastructure en fibre optique. Le SAN offre une gestion centralisée du stockage, garantit la disponibilité des données avec des fonctionnalités de redondance, assure une qualité de service élevée, et permet une évolutivité flexible en termes de bande passante et de capacité de stockage. Il est souvent privilégié pour les applications critiques nécessitant une performance optimale et une disponibilité élevée.



Comparaison SAN – NAS

➤ **Évolutivité :**

SAN : Autorise une croissance modulaire de la bande passante (BP) grâce à des commutateurs FC. Pas de limite liée à la BP d'un réseau Ethernet.

NAS : Moins flexible en termes de bande passante. Dépend de la BP du réseau Ethernet.

➤ **Capacité :**

SAN : Facile d'augmenter la capacité en ajoutant de nouveaux systèmes de stockage via des commutateurs FC. Pas de complexité liée aux limitations de connectivité des liaisons SCSI parallèles.

NAS : Moins flexible pour augmenter la capacité. Dépend souvent des capacités du réseau Ethernet.

➤ **Qualité de service (QoS) :**

SAN : Garantit la qualité de service avec un débit fixe par lien en fibre optique.

NAS : Repose sur le réseau Ethernet, offrant moins de garanties en termes de QoS.

➤ **Disponibilité :**

SAN : Assure la redondance du stockage, essentielle pour des applications critiques.

NAS : Ne propose pas toujours une redondance, moins adapté aux applications sensibles à la disponibilité.

➤ **Fonction de partage de données :**

SAN : Offre un partage physique, en lecture, et coopératif, permettant une flexibilité d'utilisation.

NAS : Peut offrir un partage physique, mais moins adapté pour une coopération simultanée sur les mêmes données.

➤ **Hétérogénéité :**

SAN : Complexité accrue de l'interopérabilité, notamment en raison de la diversité du matériel de stockage.

NAS : Plus adapté aux environnements hétérogènes grâce à l'utilisation de protocoles spécifiques aux OS.

➤ **Coût :**

SAN : Plus coûteux en raison de l'infrastructure en fibre optique.

NAS : Plus abordable, adapté aux petites entreprises avec des besoins de stockage modérés.

➤ **Administration centralisée :**

SAN : Permet une gestion centralisée et cohérente du stockage, indépendamment des serveurs et de la localisation.

NAS : Peut nécessiter une gestion plus dépendante des serveurs et du système d'exploitation.

6. Les types de supports de stockage

Bande magnétique

La bande magnétique est un support de choix pour la sauvegarde car c'est peu cher, facilement transportable et offrent de grandes capacités de stockage.

▪ ***DLT (Digital Linear Tape) et SDLT (Super Digital Linear Tape)***

Elle peut accepter des cartouches allant jusqu'à 800 Go et peut offrir des taux de transfert allant jusqu'à 120Mo/s.

▪ ***LTO (Linear Tape-Open)***

L'avantage du LTO est qu'il est open-source et peut être compatible avec les autres technologies de stockage. Cette technologie peut proposer une capacité de 1.5To pour un taux de transfert de 140Mo/s

Disque dur

- Le disque supporte les systèmes de protections RAID capables d'assurer une restauration rapide voire quasi-automatique d'un disque en panne.
- Les disques autorisent une lecture partagée des données, plus rapide que les lectures séquentielles imposées par les bibliothèques de stockage sur bandes. Les restaurations sont donc plus rapides.
- Les disques offrent des temps d'accès (temps moyen nécessaire pour placer la tête de lecture sur un secteur quelconque d'un disque), de lecture et d'écriture très performants. Les temps d'accès y sont de quelques millisecondes contre quelques secondes (voire dizaine de secondes) pour accéder aux informations placées sur une bande magnétique. Avec de tels temps d'accès, les disques sont bien adaptés à la sauvegarde incrémentale ce qui permet d'optimiser la capacité de stockage en minimisant le volume de données archivées. Pour caractériser les disques, on utilise plutôt le temps de transfert qui correspond au temps mis par les données pour être transférées entre le disque dur et l'ordinateur par le biais de son interface.

Au niveau des inconvénients, on a :

- Le prix.
- Les disques se transportent moins aisément, le stockage hors site est difficile.
- Les utilisateurs n'ont pas la certitude que l'archivage de leurs fichiers a abouti.
- Il a une moins bonne la durée de vie (10 ans en moyenne) contre 100 ans pour les médias optiques ou 30 ans pour les bandes magnétiques. Attention que l'on ne parle ici que de la durée de vie, au niveau de la fiabilité, les médias optiques restent moins performant que les deux autres supports.

SSD (Solid State Drive)

Ce type de lecteur, encore cher actuellement, est pressenti pour remplacer les disques durs. Basé sur de la mémoire flash¹ ou DRAM, il ne contient pas de pièces mobiles ce qui le rend plus fiable et bien plus rapides en termes de temps d'accès. Les vitesses des SSD SLC2 dépassent les 130 Mo/s en lecture et arrivent presque à 100 Mo/s en écriture.

Les supports optiques

Les supports optiques, tels que les CD-R, CD-RW, DVD-R, DVD-RW, DVD+R, Blu-ray, etc., sont composés de disques en matière synthétique avec une ou plusieurs couches sensibles sur une face. Les données sont gravées et lues optiquement via un laser. Certains disques, comme les magnéto-optiques (MO), combinent les technologies optiques et magnétiques. Bien que l'archivage des données sur ces supports soit lent, la restauration est généralement rapide. Les médias optiques offrent un stockage hors site facile, des copies simples et une capacité allant de 650 Mo (CD-R) à 50 Go (Blu-ray). Leur durée de vie prévue est élevée, bien que la confirmation à long terme soit difficile à établir.

Disquette

Elles ont une capacité trop limitée pour le stockage et sont peu fiables. Ses caractéristiques sont lentes, ont une faible durée de vie, sont bon marché, sont rechargeables et très portables.

7. Commandes

- a. Compression d'un fichier : `gzip filename.txt`
- b. Décompression d'un fichier : `gunzip filename.gz`
- c. Compression d'un fichier : `zip -r filename.zip files`
- d. Décompression d'un fichier : `unzip filename.zip`
- e. Créer une archive : `tar -cvf archive.tar fichier1 fichier2 fichier3`
- f. Extraire des fichiers d'une archive : `tar -xvf archive.tar`
- g. Afficher le contenu d'une archive : `tar -tvf archive.tar`
- h. Ajouter des fichiers à une archive existante : `tar -rvf archive.tar nouveau_fichier`

- i. Extraire dans un répertoire spécifique : `tar -xvf archive.tar -C /chemin/vers/repertoire`
- j. Copier le contenu d'un répertoire vers un autre tout en créant une archive : `rsync -a [SRC] [DEST]`

Chapitre 7. Installer des programmes

1. Package vs dépôt compilé

Un package est une archive qui regroupe des fichiers nécessaires à l'installation d'un logiciel. Il contient les exécutables, bibliothèques, configurations, et autres dépendances requises par le logiciel. Sous Linux, les distributions utilisent des gestionnaires de paquets comme APT (Debian, Ubuntu) ou YUM (RedHat, CentOS) pour faciliter l'installation, la mise à jour et la suppression des packages. Les packages RPM (RedHat Package Manager) contiennent une version précompilée d'un programme. L'outil de gestion de packages permet d'installer, de désinstaller de tels programmes.

Un dépôt compilé est un référentiel de logiciels où les packages ont été préalablement compilés, c'est-à-dire transformés en langage machine exécutable. Cela simplifie le processus d'installation en évitant la compilation sur la machine de l'utilisateur. Les utilisateurs peuvent installer des logiciels depuis ces dépôts sans avoir besoin de compiler le code source eux-mêmes.

Chapitre 8. Gestion des processus

1. Programme vs processus

Un processus désigne un programme chargé en mémoire et en cours d'exécution. Ainsi, à chaque fois que vous exécutez un programme, un processus est créé. Il est dynamique et représente l'exécution réelle des instructions du programme en mémoire et chaque processus a son propre espace mémoire et ses ressources dédiées, ce qui lui permet de s'exécuter indépendamment des autres processus. En résumé, le processus est l'entité dynamique résultant de l'exécution de ce programme en mémoire, avec son propre espace et contexte d'exécution.

Un programme est une séquence d'instructions écrites dans un langage de programmation spécifique. C'est une entité statique qui réside dans le stockage (disque dur, SSD, ...) de l'ordinateur. Un programme n'est pas en cours d'exécution, il représente simplement le code sources, les fichiers exécutable ou les scripts.

2. Gestion des processus

Dès qu'un programme (une commande) est démarré, le noyau Linux crée un (ou plusieurs) processus qui exécute les instructions de ce programme. Ce processus peut transiter par

différents états qu'il exécute (état actif), attend que le noyau lui alloue le processeur (état prêt) ou qu'un évènement se produise (état en attente). Pour chaque processus en cours d'exécution, l'OS stocke un certain nombre d'informations notamment dans la table des processus :

- Le numéro qui identifie le processus (Process Identification ou PID).
- Le numéro de processus parent (Parent Process Identification ou PPID).
- Le numéro de l'utilisateur qui a demandé l'exécution (User ID ou UID).
- Le numéro de groupe (Group ID ou GID).
- La durée de traitement utilisée par le processus (temps CPU).
- La priorité du processus.
- Une référence au répertoire de travail courant.
- Une liste des fichiers ouverts.

Numéro du processus (Process Identification ou PID)

Linux est multi-tâches, chaque processus associé à ces tâches doivent pouvoir se différencier. C'est pourquoi le système leur affecte un numéro spécifique de processus : le PID. Ce numéro permet d'identifier un processus sans ambiguïté. Un PID est associé à un programme lorsqu'il est exécuté et désassocié lors de son arrêt. Le PID d'un processus, après un redémarrage, ne sera pas le même que le précédent. Ce numéro permet de stopper un programme s'il ne répond plus.

Processus parent

Chaque processus peut lui-même créer des processus (child processus). Ces processus enfant/parent sont liés, il faut donc que les enfants connaissent leur origine. Tous ces processus enfant obtiennent, dès lors, le numéro d'identification de leur processus parent (PPID).

Exception : lors du démarrage du système, il y a un processus qui se crée (pseudo-processus) portant le numéro 0 et un autre qui se nomme init qui lui porte le numéro 1. Le processus 0 est particulier car il est responsable de tous les processus en cours dans le système.

Numéro d'utilisateur et de groupe (UID et GID)

Les processus, tout comme les utilisateurs, nécessitent une gestion précise des droits d'accès au système. Chaque processus reçoit à sa création un numéro d'utilisateur et un numéro de groupe. Ces identifiants permettent au système de vérifier les permissions du processus, notamment son accès à des fichiers. Généralement, les numéros d'utilisateurs et de groupes sont hérités par le processus enfant de son processus parent.

Durée de traitement et priorité

Sur un système, le nombre de processus est généralement supérieur au nombre de processeurs disponibles. Afin d'éviter la monopolisation du processeur par un seul processus, Linux utilise une gestion préemptive du multitâche. Le temps de processeur total est divisé en petites plages appelées "time slices", et chaque processus ne peut utiliser le processeur que pendant le temps

défini par le système. L'ordonnancement est effectué par le planificateur de processus, qui se base sur un système de priorité. Chaque processus a une priorité (un nombre entre -20 et 20), et le processus avec la plus haute priorité (chiffre le plus bas) est traité en premier. Pendant les périodes où un processus n'a pas accès au processeur, sa priorité augmente, et lorsqu'il est en cours d'exécution, sa priorité diminue. Ainsi, le processus ayant la plus haute priorité à un moment donné ne la conserve pas indéfiniment, assurant un traitement équitable des processus. Bien que la priorité de départ soit définie par le système lors de l'exécution, les utilisateurs peuvent la modifier, notamment avec la commande "nice".

3. Exécution d'un processus

Sous Linux, il existe quatre manières d'exécuter un processus :

- En avant-plan (foreground) : c'est le mode normal d'exécution dans une session de travail le shell n'est plus disponible car l'affichage est monopolisé par la commande et le prompt est donc indisponible.
- En arrière-plan (background) : insensible au CTRL + C, ce mode permet de garder la main sur le terminal et de lancer d'autres commandes depuis celui-ci.
- En tant que démons (daemons) : les démons désignent communément les processus détachés associés aux services du système Linux, souvent en mode client-serveur.
- En mode détaché : un processus détaché n'a plus de terminal de contrôle, ce qui signifie que le processus peut continuer de s'exécuter même si tous les shells (interfaces texte et graphique) sont arrêtés. En mode détaché, le caractère « ? » apparaît dans la colonne TTY par la commande ps qui liste les processus.

4. Héritage

Lorsqu'un processus crée un processus fils, ce dernier hérite d'un ensemble d'éléments de son père parmi lesquels on trouve :

- Le répertoire courant.
- Les variables d'environnement.
- Le umask (droits par défaut d'un fichier lorsqu'il est créé).
- Le ulimit qui fixe la taille du plus gros fichier que le processus peut créer.

5. Les signaux

C'est un chiffre qui a une symbolique bien précise pour les processus. Les signaux permettent d'agir sur l'exécution d'un processus détaché. (kill -l permet de lister les signaux possibles)

6. Traitement en tâche de fond

Lors de la saisie d'une commande dans le shell, un nouveau processus est créé pour exécuter le programme, agissant comme un processus enfant du shell. Le shell se met en attente de la fin du processus, rendant le prompt indisponible jusqu'à la terminaison de la commande. Pour exécuter plusieurs processus en parallèle sans attendre la fin de chacun, on peut les envoyer en tâches de fond en ajoutant le caractère '&' à la fin de la commande. Cela permet au shell et à ses processus enfants de fonctionner simultanément, sans coordination. Bien que les processus en tâche de fond travaillent indépendamment, ils restent liés au shell en tant que parent-enfant. À la fermeture du shell, tous les processus en tâche de fond s'arrêtent automatiquement. Si l'on souhaite que ces processus persistent en tâche de fond après la fermeture du shell, il est possible de le spécifier en leur envoyant un signal de type `nohup`.

Pour envoyer un processus en arrière-plan, certaines conditions doivent être respectées :

- Le processus envoyé en tâche de fond ne doit pas attendre d'entrée clavier pour poursuivre son exécution. Le système d'exploitation ne peut pas déterminer à quel processus en arrière-plan sont destinées ces saisies.
- Le processus envoyé en tâche de fond ne doit pas émettre ses résultats vers l'écran (sortie standard). Les sorties de ce processus pourraient entrer en conflit avec celles du processus en premier plan ou du shell, rendant l'écran difficilement lisible voire inutilisable.
- Le processus parent, généralement le shell, des processus envoyés en tâche de fond ne doit pas être arrêté, au risque d'arrêter également tous ses enfants.

Effectivement, il peut être problématique d'envoyer en arrière-plan une commande qui utilise la sortie standard (l'écran) comme sortie. Cependant, pour éviter cette limitation, il est possible de rediriger la sortie de la commande vers un autre endroit que la sortie standard, comme dans un fichier, permettant ainsi d'élargir la gamme de commandes utilisables en tâche de fond.

En plus de la sortie standard, il existe une sortie d'erreur qui est utilisée pour afficher les messages d'erreur. Dans notre exemple, un tel message sera affiché si le répertoire `/bidule` n'existe pas. Ainsi, il ne faut pas oublier de rediriger cette sortie.

Les canaux d'entrée et de sortie sont symbolisés par des chiffres :

- Le canal d'entrée standard (clavier) est représenté par le chiffre 0.
- Le canal de sortie standard (écran) est représenté par le chiffre 1.
- Le canal d'erreur standard est représenté par le chiffre 2.

Remarque :

Le fichier `/dev/null` est utilisé comme une poubelle, toutes les données y arrivant sont envoyées dans le néant.

7. Priorité d'un processus

Chaque processus se voit affecter un numéro qui correspond à une priorité. Lorsqu'un programme est démarré, le processus engendré a une priorité maximale. Plus le processus occupe de temps d'exécution au niveau du processeur, plus ce numéro de priorité baisse. À l'inverse, moins il en occupe plus son numéro de priorité augmente.

8. Commandes

- a. Lister les signaux possibles : **kill -l**
- b. Tuer un processus : **kill [PID]**
- c. Passer un processus en tâche de fond : **bg**
- d. Reprendre un processus arrêté en arrière-plan : **fg**
- e. Lister tous les processus : **ps**
- f. Afficher les processus sous forme d'une arborescence : **pstree**
- g. Afficher et classer les processus actifs (cpu – mémoire – temps) : **top**
- h. Démarrer un processus avec une priorité définie : **nice**
- i. Changer la priorité d'un processus en cours d'exécution : **renice**
- j. Rediriger une commande vers un fichier : **utilisation d'un chevron « > » pour effacer l'intérieur du fichier puis écrire dans le fichier et utilisation de deux chevrons « >> » pour écrire dans le fichier sans supprimer le reste du fichier**
- k. Mettre une commande en arrière-plan : **utilisation de & à la fin de la commande**
- l. Lister les processus que l'utilisateur a envoyés s'exécuter à l'arrière-plan : **jobs**

Chapitre 9. Gestion des ressources – partie 1

1. À quoi servent la commande crontab et le service crond ?

La commande crontab est un planificateur de tâches (ordonnancement des travaux), elle contient une liste de commandes qui doivent être exécutées à intervalle régulier. Elle permet ainsi d'automatiser une série de tâches comme la sauvegarde journalière de documents ou le nettoyage de fichiers journaux.

Ce service est chargé de faire exécuter, par le système, toutes tâches définies et planifiées dans la table crontab. Il génère des comptes rendus après leur exécution. Par défaut, le démon crond est lancé au démarrage et contrôle toutes les minutes les fichiers présents dans le répertoire /var/spool/cron, /etc/cron.d ainsi que le fichier /etc/crontab, pour voir si des tâches doivent être exécutées. Si un fichier est modifié, on ne doit pas redémarrer le service pour la vérification de ceux-ci.

La commande crontab contient 6 champs. 5 concernent la période à la laquelle la commande doit s'exécuter et la 6^{ème} concerne la commande à utiliser.

-  Le 1^{er} champ concerne les minutes (0-59).
-  Le 2^{ème} champ concerne les heures (0-23).
-  Le 3^{ème} champ concerne les jours du mois (1-31).
-  Le 4^{ème} champ concerne les mois (1-12).
-  Le 5^{ème} champ concerne les jours de la semaine (0 – 6, 0 étant dimanche).

Remarque :

Vous pouvez utiliser une liste séparée par des virgules (par exemple, 1,3,5 pour janvier, mars, mai) et un intervalle à l'aide de deux chiffres séparés par un tiret (par exemple, 1-5 pour les jours de la semaine signifie du lundi au vendredi). Le caractère * représente toutes les valeurs possibles (par exemple, * dans le champ des minutes signifie chaque minute chaque jour). En outre, des combinaisons de symboles sont possibles, telles que */5 (dans le champ des minutes, toutes les 5 minutes) et 0-23/3 (dans le champ des heures, toutes les 3 heures).

2. Qu'est-ce que Anacron ?

L'exécution des tâches programmées dépend du fonctionnement continu de la machine, ce qui peut être compromis par des arrêts inattendus. Pour pallier cela, le service anacron intervient en vérifiant et en exécutant les tâches planifiées dès que la machine est remise en route. Contrairement à crond, anacron n'est pas un démon permanent, nécessitant un démarrage spécifique et s'arrêtant une fois ses tâches terminées. Son automatisaion peut être assurée en l'incluant dans un script de démarrage.

3. La commande at

Cette commande permet définir la date d'exécution d'une tâche donnée. Contrairement à crontab qui gère les tâches périodiques, avec *at* la tâche ne s'effectue qu'une seule fois. Pour cela, le démon *atd* doit être actif, c'est lui qui vérifie chaque minute si une tâche doit être effectuée. Comme pour crontab, les tâches sont envoyées en arrière-plan pour leur exécution, il y a donc lieu de prévoir des redirections dans le cas où les tâches utilisent les entrées/sorties standards.

4. Quotas d'espace disque

C'est quoi ?

Les quotas s'appliquent sur un système de fichiers et sont de deux types :

- Les quotas qui définissent le nombre maximal de fichiers (en nombre d'inodes) qu'il est possible de créer.
- Les quotas qui définissent la taille maximale (en nombre de blocs) qui peut être occupée.

Les quotas peuvent être fixés pour un utilisateur ou pour un groupe d'utilisateurs. Leur but est d'obliger les utilisateurs à mieux gérer leur espace et les empêcher de saturer une partition. Plusieurs possibilités peuvent s'offrir à l'administrateur lorsqu'un utilisateur (ou un groupe) dépasse la taille maximale fixée par le quota.

- Soit l'utilisateur est simplement prévenu qu'il dépasse son quota et l'administrateur lui fait confiance pour qu'il nettoie ses fichiers.

- Soit l'utilisateur est prévenu qu'il est proche de son quota maximum lorsqu'il atteint une certaine limite (limite douce). Il peut cependant continuer de créer de nouveaux fichiers jusqu'à atteindre son quota (limite dure).

Les quotas sont très utilisés, notamment pour les serveurs de messagerie.

Activation des quotas

Pour activer les quotas, il faut monter les systèmes de fichiers avec l'option `usrquota` et/ou `grpquota`. Cela peut être fait manuellement avec la commande `mount` ou dans le fichier `/etc/fstab`.

Exemple :

Supposons que le point de montage de la partition `/dev/hda3` soit `/home`. Pour configurer le fichier `/etc/fstab`, on ajoute l'option `usrquota` (et/ou `grpquota`) sur la ligne qui configure le montage de `/home`.

```
/dev/hda3 /home ext2 default,usrquota,grpquota 1 2
```

Les fichiers contenant les définitions des quotas (`quota.user` & `quota.group`) peuvent être créés automatiquement grâce à la commande `quotacheck`. Cette commande vérifie la présence et la cohérence des informations de gestion des quotas.

```
#!/usr/sbin/quotacheck -avug
```

L'activation des quotas est ensuite lancée par la commande `quotaon` (`quotaoff` dans le cas contraire). Pour les activer automatiquement lors du démarrage du système, il faut ajouter dans un fichier d'initialisation (situé généralement dans `/etc/rc.d`) :

```
#!/usr/sbin/quotaon -avug
```

L'attribution d'un quota à un utilisateur se fait avec la commande `edquota -u [user] ou -g [group]`. Deux limites peuvent se fixer :

- La limite douce : lorsque cette limite est atteinte ou dépassée, un message d'avertissement est affiché lors de chaque nouvelle allocation de fichier ou de bloc.
- La limite dure : lorsque cette limite est atteinte (elle ne peut être dépassée), il est impossible à l'utilisateur de créer de nouveaux fichiers ou d'allouer de nouveaux blocs.

La limite douce se transforme en limite dure quand elle a été atteinte ou dépassée depuis un temps prédéfini appelé « la période de grâce ».

L'administrateur ou le sudo-user peut regarder en détail les quotas associés à un ou plusieurs systèmes de fichiers avec la commande `repquota -avug` contrairement à l'utilisateur simple qui peut utiliser que la commande `quota`.

5. Commandes

- a) Afficher le fichier texte de crontab (explication de la commande dans une des sections se trouvant dans ce chapitre) pour le modifier (niveau utilisateur) : **crontab -e**
- b) Faire une commande qui ne s'exécutera une fois à un temps donné : **at [time]** (ex : **at now + 1 min** ou **at 10 :00 PM tomorrow** ou **at 14 :00 -f chemin/vers/votre/script**) puis faire **enter**, vous aurez **at>** [faire votre commande ici] puis faites **CTRL + D**.
- c) Activer les quotas : **quotaon -avug**
- d) Désactiver les quotas : **quotaoff -avug**
- e) Vérifier les quotas : **quotacheck -avug**
- f) Modifier les quotas d'un utilisateur et/ou d'un groupe : **edquota -u [user] ou -g [group]**
- g) Visualiser les statistiques d'un quota (coté utilisateur) : **quota**
- h) Visualiser l'ensemble des quotas fait sur plusieurs utilisateurs et/ou systèmes de fichiers (coté administrateur) : **repquota -avug**
- i) Afficher des informations sur le système : **uname -a**
- j) Afficher et/ou configurer le nom d'hôte du système : **hostnamectl**
- k) Afficher l'utilisation de la mémoire : **free -h**
- l) Exécuter une commande chaque n time : **watch -n [temps en secondes] [commande]**
- m) Afficher seulement les processus actifs d'un utilisateur : **top → i**
- n) Classer les processus par ordre décroissant d'utilisation de mémoire : **top → shift + F → n**
- o) Lister les processus démarrés par un utilisateur : **top -u [user]**
- p) Visualiser les informations de l'utilisation des ressources pendant un temps t et n fois : **vmstat [temps] [nombre_de_fois]**
- q) Montrer le temps de connexion du système et le nombre d'utilisateur(s) : **uptime**
- r) Actualiser la base de données locate : **updatedb**
- s) Évaluer les ressources et le temps employé par une commande : **time [commande]**
- t) Visualiser les commandes qui vont être exécutées : **atq**
- u) Empêcher l'exécution d'une future commande : **atrm**
- v) Redémarrer un service : **systemctl restart [nom_du_service]**
- w) Désactiver la surveillance des quotas : **quotaoff -ug /[partition]**

Chapitre 10. Gestion des ressources – partie 2

1. Pourquoi surveiller les ressources ?

Si les ressources ne sont pas bien utilisées, elles peuvent ralentir voire rendre le système inutilisable. Il faut réagir en urgence car un problème peut prendre du temps à être corrigé et il faut être proactif – surveiller le système avant les problèmes pour connaître l'utilisation des ressources dans un fonctionnement normal.

2. Quelles sont les ressources à surveiller ?

Les ressources principales à surveiller sont :

- Le CPU.
- La mémoire.
- Les entrées/sorties.
- L'espace disque.

Il faut aussi, avant tout problème, respecter la configuration demandée par les OS et les éditeurs de logiciels. Les benchmarks peuvent aussi aider pour simuler le comportement de l'activité de l'application.

3. Surveillance du CPU

Que faire en cas de niveau élevé d'utilisation de temps CPU ?

Il ne faut rien faire si cela se produit ponctuellement et qu'il n'y a pas de problèmes → le système travaille. Cependant, s'il y a un problème de performances + utilisation élevée de temps CPU l'insuffisance de cycle CPU est sans doute un facteur qui contribue au problème.

Que faire lorsque des conflits de ressources CPU sont la source d'un goulot d'étranglement ?

Il faut favoriser certaines tâches avec la priorité des processus, réduire la consommation du CPU (déplacer une partie de la charge sur un autre système et exécuter certaines tâches plus tard dans la journée) et modifier la méthode d'ordonnancement des processus (si le système le permet). Vous pouvez utiliser le mécanisme de pagination (swapping). Lorsque les processus demandent plus de mémoire que ce qui est disponible physiquement dans la RAM, le système d'exploitation utilise le "swapping". Cela implique le transfert de parties inactives des processus de la RAM vers le disque pour libérer de l'espace. Ainsi, même si la mémoire physique est limitée, le système peut gérer les demandes de mémoire en échangeant des données entre la RAM et le disque.

Ce mécanisme a comme avantages :

- Il rend possible l'utilisation de mémoire virtuelle par le biais de laquelle la mémoire utilisée par un processus peut excéder la mémoire physique disponible.

- Cet état est rendu possible par le fait que les processus n'ont pas constamment besoin de toute la mémoire.

En revanche, si la mémoire est insuffisante et la gestion des priorités est mal étudiée, le système risque de passer beaucoup de temps à paginer plutôt qu'à exécuter les processus.

4. Surveillance de la mémoire

Gestion de l'espace de pagination

Il faut faire en sorte d'avoir une bonne gestion de l'espace de pagination :

- Pour un environnement mono-utilisateur, où un seul utilisateur interagit avec le système à la fois, l'espace de pagination peut être configuré pour être équivalent à 1 à 2 fois la taille de la mémoire physique. Cela permet de gérer efficacement les besoins en mémoire de cet utilisateur.
- Lorsqu'il y a un besoin accru de mémoire, par exemple dans des situations où plusieurs utilisateurs ou des applications gourmandes en mémoire sont en cours d'exécution en même temps, il peut être recommandé d'allouer un espace de pagination plus importante (entre 3 et 4 fois la taille de la mémoire physique).
- Idéalement, la meilleure approche est de disposer de suffisamment de mémoire physique pour répondre aux besoins des processus sans avoir recours fréquent à l'espace de pagination. Une mémoire physique abondante améliore les performances globales du système en évitant les opérations de transfert de données entre la RAM et le disque, qui sont plus lentes.

Les facteurs influençant l'espace de pagination sont les suivants :

- Les tâches qui nécessitent beaucoup de mémoire.
- Les programmes très gros.
- Un nombre important de tâches qui s'exécutent en même temps.

On l'active soit dans `/etc/fstab`, soit avec les commandes `swapon` ou `swap`.

Facteurs influençant les performances des entrées-sorties sur disque

- Répartition des disques et des contrôleurs.
- Répartition des données sur les différents disques.
- Localisation des fichiers sur le disque physique.

5. Comment résoudre un problème de performance du système ?

Poser le problème

Il faut décrire le problème de la manière la plus précise possible.

Déterminer la cause du problème

Il faut déterminer le type de preuve que vous devez recueillir et puis les récolter.

- Quand et sous quelles conditions le problème survient-il ?
- Une modification du système a-t-elle été réalisée récemment ?
- Quelles ressources affectent les performances ?
- Qu'est ce qui continue de fonctionner normalement ?
- ...

Formuler les objectifs à atteindre

Il faut formuler les objectifs à atteindre car ils peuvent aider la réalisation des modifications.

Concevoir les modifications adéquates

Il faut savoir déterminer les modifications (la partie la plus compliquée) et procéder au réglage de l'ensemble du système car les performances sont le résultat de l'interaction de tous les composants du système. Il est conseillé de procéder par des petites modifications successives car c'est généralement le plus efficace.

Surveiller

La surveillance du système permet d'évaluer si les modifications ont apporté les améliorations attendues.

Recommencer

Une modification du système entraînera de nouvelles interactions entre processus, il sera peut-être nécessaire de revenir à la première étape et recommencer.

Remarque :

Les ressources ne sont pas infinies → compromis sur la manière d'allouer et de partager les ressources (fonction de la priorité et de l'importance relatives des différentes activités).

Chapitre 11. Le noyau Linux

1. Informations générales concernant les noyaux

Rôle d'un OS (Operating System)

Il permet l'utilisation d'un ordinateur et de ses périphériques via des logiciels. Il contient un noyau et des outils système.

Le noyau linux

Il permet de faire de nombreuses choses dont :

- Il est responsable des tâches de base de l'O.S.
- Il permet la gestion de la mémoire.
- Il permet la gestion des processus pour les noyaux multi-tâches.
- Il permet la gestion de systèmes de fichiers.
- Il permet la gestion des entrées/sorties (avec des pilotes de périphériques).
- Il possède une interface avec l'espace utilisateur.
- Il gère les services réseaux (TCP/IP, PPP, Firewall, ...).

Les types de noyau

Il existe deux types de noyau :

- Noyau monolithique - il contient tous les pilotes et ne prend pas en charge les modules.
- Noyau modulaire - ce noyau possède trois options :
 - **Yes** : Si une fonctionnalité ou un pilote est configuré avec l'option "Yes", cela signifie qu'il sera intégré directement au code source du noyau. Cela rend cette fonctionnalité ou ce pilote partie intégrante du noyau, et le noyau résultant contiendra tout ce qui a été configuré avec "Yes".
 - **Module** : Si une fonctionnalité ou un pilote est configuré en tant que "Module", cela signifie qu'il sera compilé en tant que module séparé. Les modules sont des morceaux de code qui peuvent être chargés ou déchargés dynamiquement dans le noyau en cours d'exécution. Cela offre une flexibilité considérable, car le noyau peut être étendu avec de nouvelles fonctionnalités sans avoir à être recompilé entièrement.
 - **No** : Si une fonctionnalité ou un pilote est configuré avec "No", cela signifie qu'il ne sera pas inclus dans le noyau. Cette option est choisie lorsque l'utilisateur ne souhaite pas inclure cette fonctionnalité ou ce pilote spécifique dans le noyau.

Chapitre 12. LVM (Logical Volume Manager)

1. Qu'est-ce que c'est ?

C'est une couche logicielle entre le système de fichiers et les partitions. Elle permet de gérer de manière souple les espaces de stockage. Elle a de nombreux avantages tels que :

- Flexibilité pour allouer de l'espace aux applications et aux utilisateurs.
- Redimensionnement et déplacement des volumes de stockage à la demande et à chaud.
- Gestion par groupes d'utilisateurs.
- Les instantanées (snapshot).
- Fonctionnalité RAID 0 et RAID 1.

Cependant, le LVM possède aussi quelques inconvénients :

- Les volumes logiques doivent être mis en place dès la création de la partition.
- Les volumes logiques s'occupent de la partition et non du système de fichiers.
- Volumes répartis sur plusieurs disques.
- Gestion/dépannage plus compliqué.
- Les boot loader ne parviennent pas à démarrer sur une partition LVM.

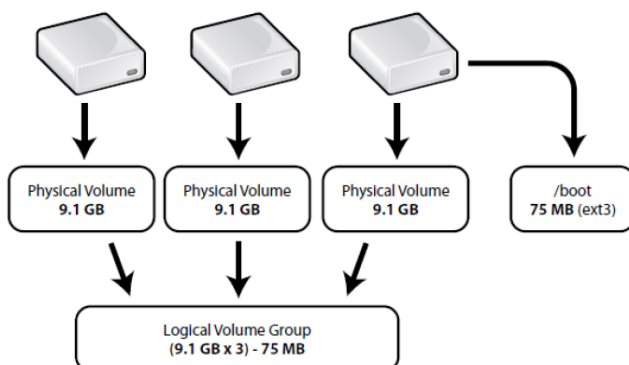
2. Les différents concepts

Physical Volume (PV)

Un disque physique ou une partition est associé à un volume physique.

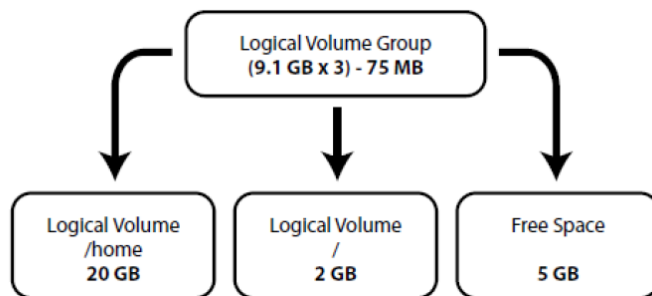
Volume Group (VG)

C'est un ensemble de volumes physiques et cet ensemble est divisé en volumes logiques



Logical Volume (LV)

C'est un équivalent logique d'une partition. Il peut contenir un système de fichiers et le LV possède son fichier dans /dev. Il peut être associé à un point de montage.



Physical Extent (PE)

C'est une unité d'allocation physique de données. Chaque PV est divisé en PE (par défaut, 1 PE = 4Mo).

Logical Extent (LE)

C'est une unité d'allocation logique de données. Chaque LV est divisé en LE. Les partitions logiques peuvent être agrandies d'un ou de plusieurs extents. La taille d'extents est la même pour tous les volumes logiques du groupe de volumes.

Chapitre 13. Démarrage et arrêt d'un système Linux et gestion des services

1. Procédure de démarrage

Il y a l'auto-amorçage, c'est-à-dire une machine a besoin d'un O.S pour fonctionner, mais elle doit aussi être capable de lancer son O.S elle-même. Mais il y a aussi une procédure simplifiée :

- 1 mise sous tension.
- Exécution des instructions du programme de chargement du BIOS.
 - Ces instructions sont stockées en mémoire ROM (mémoire morte). Elles ont pour but de :
 - Vérifier la disponibilité des ressources (POST : Power On Self Test).
 - Localiser et exécuter le programme principal de démarrage (boot loader) → Une partie de celui-ci est généralement situé dans le MBR (Master Boot Record) du disque dur.

Explication du MBR :

- 446 octets contiennent le programme de chargement (le loader).
- 64 octets décrivent les partitions primaires (limitation à 4 partitions primaires).
- 2 octets qui représentent le « magic number » → permet de vérifier la signature du secteur.

Ensuite le programme de démarrage principal (loader) se charge en mémoire. Exemple de boot loader : Grub2, Loadlin, ... Ce programme de démarrage donne la possibilité de choisir quel système d'exploitation doit être démarré. Il peut être placé à différents endroits : disquette, MBR, serveur de fichiers. Il cherche sur la table des partitions une partition active, puis il charge de boot de cette partition. Il charge le reste du programme de démarrage. Il localise le noyau et le charge.

Quand le noyau est chargé, le noyau s'exécute et s'initialise : réservation de la mémoire, prise en compte de la zone d'échange (swap), chargement des pilotes des périphériques et le montage des systèmes de fichiers, ... Le montage du système de fichiers racine « / » va permettre le lancement du premier processus : /sbin/init (dont le PID est 1) ou pour systemd : /sbin/systemd.

La suite de la procédure d'initialisation est donnée à un gestionnaire de service : il permet de les lancer, les arrêter, les relancer, d'en assurer un suivi, etc. C'est aussi un gestionnaire de session. Il permet l'utilisation du même matériel en passant d'une session à une autre sans que cela pose de problème. Il permet encore de gérer les points de montage qu'ils soient automatiques ou non.

2. GNU GRUB (Grand Unified Bootloader)

C'est un programme d'amorçage GNU qui gère la gestion du chargement des OS disponibles sur le système. Il permet à l'utilisateur de choisir quel système démarrer. Il intervient après l'allumage de l'ordinateur et avant le chargement du système d'exploitation.

3. Init vs Systemd

Init est le programme sous Unix qui lance ensuite toutes les autres tâches (sous forme de scripts). Il s'exécute comme un démon informatique alors que systemd est une alternative au démon init. Il est spécifiquement conçu pour le noyau Linux. Il a pour but d'offrir un meilleur cadre pour la gestion des dépendances entre services, de permettre le chargement en parallèle des services au démarrage, et de réduire les appels aux scripts shell.

Remarque :

systemd dispose d'une notion d'unité. Les services sont un type d'unité. D'ailleurs systemd est le système d'initialisation installé par défaut avec les distributions Arch Linux, Centos 7, Debian 8, ...

4. Les niveaux d'exécutions (runlevels)

Le run-level (niveau de fonctionnement) est un chiffre ou une lettre utilisée par le processus init des systèmes de type Unix pour déterminer les fonctions activées du système. Un système Linux offre plusieurs niveaux de fonctionnement :

0. Arrêt du système.
1. Mode maintenance.
2. Mode multi-utilisateur en console (pas de Network File System).
3. Mode multi-utilisateur en console.
4. À définir par l'utilisateur.
5. Mode multi-utilisateur avec interface graphique.
6. Arrêt et redémarrage du système.
7. Les autres sont à définir par l'utilisateur.

Il y a aussi les niveaux de fonctionnement systemd :

Init	Systemd	
0	Runlevel0.target, poweroff.target	Arrêt
1	Runlevel1.target, rescue.target	Mode utilisateur unique
3	Runlevel3.target, multi-user.target	Mode multi non-graphique
2, 4	Runlevel2(ou 4).target, multi-user.target	Idem
5	Runlevel6.target, graphical.target	Mode multi-user graphique
6	Runlevel6.target, reboot.target	Redémarrage
Emerg	Emergency.target	Shell d'urgence

5. Les grands principes de systemd

Le grand principe de systemd est de créer les canaux de communication entre les processus avant de lancer les processus des services. La synchronisation effectuée par le système permet de gérer une partie des dépendances et l'autre mécanisme de communication inter-processus est DBUS.

6. Notion d'unité

Systemd utilise une notion d'unité (« Unit »). Il y a différents types d'unité : automount, path, mount, service, socket, target, timer, wants → socket et service sont les plus utilisés. Les unités de type socket permettent à systemd de connaître les sockets à créer. Les unités de type de service permettent à systemd de connaître les services à lancer. Bien entendu, les sockets sont traités par systemd avant les services.

7. Les fichiers associés à l'utilisateur

- Le rôle de ces fichiers est de permettre de définir un environnement de travail propre à chaque utilisateur.

Nous avons quelques fichiers associés à l'utilisateur et voici quelques exemples :

- ***/etc/profile***

C'est un fichier commun à tous les utilisateurs. Il permet de configurer le shell et définit les variables d'environnement globales et programmes de démarrage. Ensuite, il exécute les scripts placés dans `/etc/profile.d/` → personnalisation des couleurs de l'écran des consoles, ...

- ***/etc/bashrc***

Il contient les alias et fonctions systèmes comme le `umask` par défaut ou la variable `PS1` qui définit le format du prompt par défaut.

- ***/etc/skel***

Ce répertoire est utilisé par `useradd` pour s'assurer que tous les nouveaux utilisateurs commencent avec la même configuration de base. Les fichiers qu'il contient sont copiés dans le répertoire personnel de l'utilisateur.

8. Fichiers de l'utilisateur

- ***~/.bash_profile***

Il contient les variables d'environnement et les programmes de lancement personnel. On peut aussi y personnaliser le prompt.

- ***~/.bashrc***

Il contient les alias et fonctions personnelles.

- ***~/.bash_logout***

Il est lu par le shell quand un utilisateur se déconnecte du système. Il permet d'exécuter un processus à la fin d'une session.

- ***~/.bash_history***

Il contient l'historique des commandes exécutées.

9. Journalisation du démarrage

La plupart des systèmes Linux sauvegardent dans un journal certains ou la totalité des messages générés pendant le démarrage. L'enregistrement des événements systèmes est géré par deux programmes : klogd et syslog. Klogd est simplement à l'écoute des messages en provenance du noyau et les envoie à syslog.

10. Commandes

- a. Afficher le compte rendu du démarrage : **dmesg**
- b. Afficher le mode de fonctionnement courant ainsi que l'heure à laquelle il a été initié :
who -r
- c. Afficher le niveau de fonctionnement par défaut : **systemctl get-default**
- d. Configurer le niveau de fonctionnement par défaut : **systemctl set-default [runlevel.target]**
- e. Changer dans la session actuelle la cible par défaut : **systemctl isolate [runlevel.target]**
- f. Installer un service : **dnf**
- g. Création d'un script de démarrage (/etc/systemd/system/[exemple].service)

[Unit]

Description= ps.log

[Service]

ExecStart=bash -x /usr/bin/monscript.sh (les scripts (monscript.sh) se mettent dans /usr/bin/)

Restart=on-failure

Type=oneshot

[Install]

WantedBy=default.target

- h. Recharger les démons : **systemctl daemon-reload**
- i. Lancer son service créé : **systemctl start monservice.service**
- j. Charger son service créé à chaque démarrage : **systemctl enable monservice.service**

Chapitre 14. Mise en réseau sous Linux

1. Les fichiers de configuration

Résolution des noms

Le fichier qui permet d'assurer la résolution des noms est `/etc/hosts`. Un autre fichier est aussi utilisé pour la résolution des noms : `/etc/resolv.conf`. Ce fichier contient le nom de domaine, l'adresse du (ou des serveurs) de nom à contacter.

Paramètres de la machine

Le fichier qui permet de fixer le nom de la machine et quelques autres paramètres réseaux est `/etc/sysconfig/network`. Le répertoire `/etc/sysconfig` est utilisé pour passer des informations de configuration à certains programmes qui démarrent.

Il y a aussi le répertoire `/etc/NetworkManager/system-connections/` qui contient le fichier de configuration réseau de la machine.

Base de données

Le fichier qui permet de contacter les bases de données et de lister les DB ainsi que les moyens d'obtenir l'information demandée est `/etc/nsswitch.conf`.

2. NFS (Network File System)

L'objectif du NFS est de permettre à des hôtes distants de monter des systèmes (répertoires, partitions) et les utiliser comme s'il s'agissait de systèmes de fichiers locaux. Il permet de centraliser les systèmes de fichiers et leur administration. De plus, il est simple d'utilisation car le client n'a pas besoin de connaître les détails réseau.

Bref historique du NFS

Il y a eu en tout 4 versions pour le NFS :

- La première et la deuxième version :
 - Connexion sans statut entre client et serveur.
 - Trafic réseau faible.
 - Versions non sécurisées.
- La troisième version :
 - Il y a eu plus de fonctionnalités :
 - Rapports d'erreurs avancés, ...
 - Gestion de la sécurité élémentaire.
 - Le système est sans état et ne permet pas la reprise sur incidents.

- La dernière version actuelle (quatrième version)
 - Gestion totale de la sécurité
 - Négociation du niveau de sécurité entre le client et le serveur.
 - Authentification forte (supporte Kerberos).
 - Les listes de contrôles d'accès (ACL).
 - Systèmes de maintenance simplifiés
 - Réplifications possibles.
 - Migration en toute transparence pour le client.
 - Reprise sous incidents
 - Intégré du côté client & serveur.
 - Compatibilité
 - NFSv4 supporte Unix et MS-Windows.

Pour résumer, le NFSv4 (la dernière version) est recommandé pour éviter des problèmes comme l'absence de chiffrement, des authentifications non sécurisées, des manques de contrôles d'accès, ...

Procédures pour mettre en place un serveur NFS (client & serveur)

▪ **Installation du service NFS (Network File System)**

- Installer le service : `dnf install nfs-utils`

▪ **Mise en place du service NFS installé**

- Activer le service : `systemctl start nfs-server.service`
- Activer le service après chaque redémarrage : `systemctl enable nfs-server.service`
- Vérifier si les RPC nécessaires à NFS sont démarrés : `rpcinfo -p`

▪ **Modifier le fichier `/etc/exports`**

Ce fichier permet de rendre possible le partage.

- Éditer le fichier : `vi-vim-nano /etc/exports`

Structure du fichier `/etc/exports` :

Répertoire	Hôte1(option1,option2)	hôte2(option1, ...)
/home	10.1.70.199(ro)	PC12(rw)

Quelques exemples d'option :

all_squash : Cette option fait en sorte que tous les UID et GID soient mappés sur l'utilisateur anonyme (anonymous user) lors des opérations de lecture et d'écriture. Cela signifie que, du

point de vue du serveur NFS, tous les utilisateurs sur le client sont traités comme s'ils étaient l'utilisateur anonyme. C'est utile pour des répertoires publics où l'on veut restreindre l'accès.

root_squash : Cette option spécifique fait en sorte que les requêtes provenant de l'UID/GID 0 (root) soient mappées sur l'UID/GID anonyme. Cela renforce la sécurité en évitant que l'utilisateur root du client n'ait des privilèges élevés sur le serveur NFS.

no_root_squash : Cette option désactive le "root squashing", ce qui signifie que l'UID/GID 0 (root) du client conserve ses privilèges sur le serveur NFS. Cela peut être utile dans des scénarios où l'on veut autoriser un client à utiliser ses privilèges root sur le serveur.

anonuid et anongid : Ces options permettent de spécifier explicitement l'UID et le GID de l'utilisateur anonyme. Par exemple, on pourrait utiliser ces options pour définir l'UID et le GID de l'utilisateur anonyme sur 150.

async : cette option peut être utilisée pour augmenter les performances du système car il répond immédiatement aux opérations d'écriture et lecture sans attendre qu'elles soient effectivement écrites sur le support de stockage. Il peut y avoir une corruption des données si le système crash et que les informations ne soient pas écrites de manière permanente sur le support.

sync : Au contraire de async, le système attendra que les opérations d'écriture/lecture soient écrites sur le support de manière permanente. Il y a donc une plus grande fiabilité des données mais les performances du système seront touchées.

...

Remarque :

Un hôte peut être renseigné avec un nom de domaine, un nom d'hôte, une adresse IP. Nous pouvons aussi utiliser les caractères * (tout) mais il ne faut pas l'utiliser avec les adresses IP. Les options de /etc/exports peuvent se trouver en faisant la commande : **man exports**. Exporter un système de fichiers veut signifier rendre le système de fichiers accessibles à distance.

- Exporter tous les systèmes de fichiers se trouvant dans /etc/exports : **exportfs -a**

- **Recharger le service NFS**

- Recharger le service NFS : **systemctl restart nfs-server.service**

- **Configurer le client NFS (sans /etc/fstab)**

- Monter le répertoire : **mount -t nfs**
[nom_du_serveur_NFS_ou_adresse_IP]:/[répertoire_que_vous_voulez_partager]
/[chemin_de_montage_local]

- **Configurer le client NFS (avec /etc/fstab)**

- Éditer le fichier /etc/fstab : **vi-vim-nano /etc/fstab**
- Ce que vous devez mettre dans le fichier fstab : **[IP_serveur_NFS]:/[répertoire]**
/[chemin_du_montage] [type] [opt_montage] [dump] [fsck_order]

« **dump** » : Il indique si le système doit inclure ce système de fichiers lors de la sauvegarde automatique (par exemple, lors de l'utilisation de la commande `dump`). Les valeurs possibles sont :

- 0 : Aucune sauvegarde automatique ne sera effectuée pour ce système de fichiers.
- 1 : Le système de fichiers sera inclus dans les sauvegardes automatiques.

« **fsck_order** » : Il détermine l'ordre dans lequel les systèmes de fichiers seront vérifiés au démarrage du système à l'aide de la commande **fsck**. Les valeurs possibles sont :

- 0 : Aucune vérification automatique du système de fichiers au démarrage.
- 1 : Le système de fichiers sera vérifié en premier.
- 2 : Le système de fichiers sera vérifié après ceux ayant la valeur 1.

Quelques exemples d'option de montage :

noexec : N'autorise pas l'exécution de binaires sur le système de fichiers monté.

nosuid : Empêche l'utilisation des bits set-user-identifier ou set-group-identifier.

nfsvers=2 ou **nfsvers=3** : Spécifie la version du protocole NFS à utiliser.

Voir **man mount** pour d'autres options...

- Recharger les démons : **systemctl daemon-reload**
- Monter tous les systèmes de fichiers : **mount -a**
- Mettre les permissions correctes sur les dossiers : **chmod ...**

▪ **Mise en place et activation du service autofs**

Avec autofs, tout programme ou utilisateur qui entrerait dans un répertoire qui est assigné à un périphérique provoque l'attachement du périphérique à ce répertoire (montage). Ensuite, autofs les démontent automatiquement après une période d'inactivité. Ce service permet de limiter les ressources employées.

Écrire dans le fichier `/etc/exports`

(Revoir la structure du fichier au-dessus)

Recharger le service `nfs-server`

- `systemctl restart nfs-server`

Installer le service autofs

- `dnf install autofs`

Modifier le fichier `/etc/auto.master`

Ajouter cette structure à la fin du fichier :

`[point_de_montage] /etc/[fichier_conf] [options]`

Exemple :

`/mnt /etc/auto.nfs --timeout=60`

Le fichier `[fichier_conf]`, vous allez devoir le créer puis l'éditer.

Structure du fichier `[fichier_conf]` :

`[mot_clé] [même options que dans /etc/exports] [IP_serveur_NFS]:/[répertoire_à_partager]`

Exemple :

`share -rw,async,all_squash 192.168.200.3:/nfs/share`

`[mot_clé]` → permet de créer le dossier dans le point de montage. Dans l'exemple ci-dessus, le point de montage se fait dans `/mnt/[mot_clé]` donc `/mnt/share`.

Recharger le service autofs

- `systemctl restart autofs`

Regarder si tout fonctionne

- `ls /[point_de_montage]/[mot_clé]`

3. Principe RPC (Remote Procedure Call)

Un RPC est un mécanisme qui permet à un programme de demander l'exécution d'une procédure (fonction ou méthode) sur un autre espace d'adressage (généralement sur une machine distante) comme s'il s'agissait d'une procédure locale. Cela permet d'abstraire la communication entre processus ou machines, facilitant le développement d'applications distribuées.

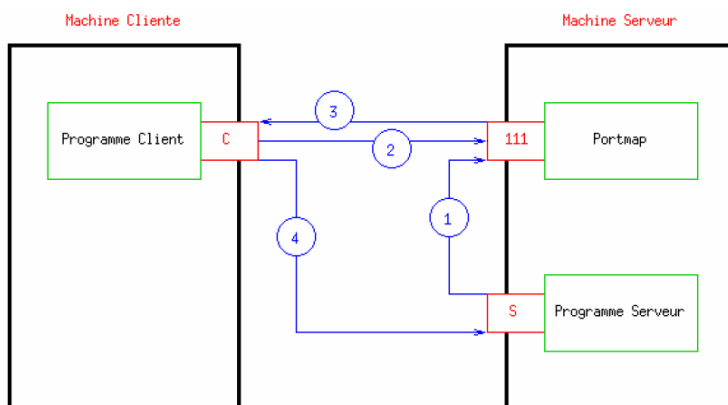
Voici les principes de base du RPC :

- **Client-Server Interaction** : Un RPC implique généralement un modèle client-serveur où un client fait une demande à un serveur distant pour exécuter une procédure spécifique.
- **Interface Définie** : Les procédures qui peuvent être appelées à distance sont décrites dans une interface, souvent sous forme d'IDL (Interface Definition Language). Cette interface définit le nom des procédures, leurs paramètres et leurs types de retour.
- **Marquage des Paramètres** : Les paramètres de la procédure à appeler sont emballés dans une structure de données spécifique (encodage des paramètres). Cela permet de transférer les données entre le client et le serveur, même s'ils n'utilisent pas la même représentation interne.

- **Invocation Distante** : Le client invoque la procédure à distance comme s'il s'agissait d'une procédure locale. Le serveur reçoit cette demande et la décode pour obtenir les paramètres.
- **Exécution Locale** : Le serveur exécute la procédure localement, en utilisant les paramètres fournis par le client.
- **Résultats Retournés** : Une fois la procédure terminée, le serveur renvoie les résultats au client. Cela implique également l'encodage des résultats pour le transfert.
- **Gestion des Erreurs** : Les mécanismes RPC incluent généralement des méthodes pour gérer les erreurs, telles que la perte de connexion ou des erreurs spécifiques à la procédure.
- **Communication Basée sur un Protocole** : La communication entre le client et le serveur est gérée par un protocole, qui définit la manière dont les données sont empaquetées, envoyées et reçues.

Attention, un client distant ne connaît pas les services RPC qui fonctionnent sur le serveur & sur quel port ces services sont installés. C'est pour cela qu'il existe le démon portmap qui permet de mapper les demandes RPC aux bons services en faisant la correspondance entre le numéro de service RPC et celui du port TCP & UDP associé.

Fonctionnement du RPC



1. Au démarrage, le programme serveur (Network File System par exemple) s'enregistre auprès du serveur RPC (démon portmap ou rpcbind), il annonce le numéro de port qu'il contrôle et les numéros de programmes RPC qu'ils servent.
2. Le client demande au serveur RPC le numéro de port du service auquel il veut accéder.
3. Réponse du serveur RPC qui communique le numéro de port.
4. Le client peut envoyer sa requête au serveur.

Avantages et inconvénients du RPC

▪ Avantages du RPC :

- **Abstraction de la Communication** : Les RPC permettent aux développeurs de traiter les appels de procédures à distance de la même manière que les appels de procédures locales, offrant ainsi une abstraction de la communication réseau.

- **Réutilisation de Code** : Les procédures peuvent être écrites et testées localement, puis réutilisées à distance sans modification du code source.
 - **Interopérabilité** : Les RPC facilitent l'interopérabilité entre différentes plates-formes, langages de programmation et environnements d'exécution.
 - **Productivité** : Le développement d'applications distribuées peut être plus rapide et plus productif avec l'utilisation de RPC, car il évite la complexité liée à la gestion manuelle de la communication inter-processus.
 - **Séparation des Responsabilités** : Les développeurs peuvent se concentrer sur l'implémentation de la logique métier, tandis que la gestion de la communication réseau est prise en charge par le mécanisme RPC.
- **Inconvénients du RPC :**
- **Complexité** : La mise en œuvre et la gestion d'un mécanisme RPC peuvent être complexes, en particulier lorsqu'il s'agit de gérer des problèmes tels que la concurrence, la gestion des erreurs et la synchronisation.
 - **Dépendance à la Plate-forme** : Certains mécanismes RPC peuvent introduire une dépendance à une plate-forme spécifique, rendant le code moins portable.
 - **Surcharge Réseau** : L'utilisation intensive de RPC peut entraîner une surcharge réseau, en particulier dans le cas de petits appels de procédures fréquents.
 - **Performance** : Les RPC peuvent introduire un surcoût de performance par rapport aux appels de procédures locaux en raison des opérations supplémentaires nécessaires pour la sérialisation, la transmission et la désérialisation des données.
 - **Gestion des Erreurs Complexes** : La gestion des erreurs dans un environnement RPC peut être complexe, en particulier lorsque des erreurs réseau ou des défaillances de communication se produisent.
 - **Sécurité** : La conception sécurisée des RPC peut être complexe, et des vulnérabilités peuvent être exploitées si la sécurité n'est pas correctement implémentée.

4. Commandes

- a. Configuration des interfaces réseaux : **nmtui**
- b. Afficher l'état des interfaces réseaux : **nmcli**, **ip address** ou **ifconfig**
- c. Envoyer une requête ICMP à un périphérique réseau : **ping**
- d. Affichez l'état de la table de routage locale : **route**, **netstat**
 - U (la route est active = up)
 - H (la cible est un hôte)
 - G (utilise comme passerelle)
 - R (rétablit la route pour le routage dynamique)
 - D (dynamiquement configurée par le démon ou par redirect)
 - M (modifiée par le démon de routage ou par redirect)
 - ! (rejète la route)
- e. Interroger les noms de serveurs DNS : **dig**
- f. Interroger les noms de serveurs DNS (moins complet que la commande dig) : **nslookup**
- g. Convertir des noms de domaine en IPv4 et vice-versa : **host**

- h. Visualiser les informations sur la route suivie pour atteindre un hôte : **tracpath**
- i. Afficher le nom de l'ordinateur et/ou le changer : **hostname**
- j. Sniffer le trafic réseau passant sur une interface donnée : **tcpdump**
- k. Manipuler le cache ARP : **arp**

Chapitre 15. Introduction au shell BASH et scripting shell

1. Qu'est-ce qu'un shell ?

C'est un interpréteur de commandes, c'est-à-dire que c'est un programme qui se charge de lire et d'exécuter les commandes dans l'environnement utilisateur. Il utilise un environnement de programmation qui permet de définir des variables, des fonctions, des instructions complexes et des programmes complets. Cet environnement dispose de tous les mécanismes nécessaires pour une programmation structurée. Il peut utiliser des scripts shell qui sont des fichiers « texte » qui contiennent des instructions exécutables et peut aussi utiliser des scripts récursifs.

→ Permet d'automatiser certaines tâches.

2. BASH (Bourne Again Shell)

C'est l'évolution du Bourne Shell (BSH) et a son propre langage de programmation.

Les redirections (>)

Permet de rediriger les données d'un flux vers un autre flux. Il existe trois types de flux :

- Les flux d'entrées (0 – stdin - clavier).
- Les flux de sortie (1 – stdout – écran).
- Les flux d'erreurs (2- stderr – écran).

▪ **Les redirections de données en sortie**

Permet d'enregistrer dans un fichier, les données écrites par un processus dans un de ses descripteurs de fichier (utilisation du > [effacer et écrire] ou >> [juste écrire à la fin du fichier]).

▪ **Les redirections de données en entrée**

Injecter des données provenant d'un fichier dans un descripteur de fichier d'un processus (utilisation du <).

Les pipes (|)

Un pipe permet d'injecter le résultat d'une commande dans le flux d'entrée standard d'une autre commande, sans passer par un fichier intermédiaire.

Exemple :

```
ps | grep bash
```

Les commandes multiples

Utilisation du « ; » pour exécuter plusieurs commandes en même temps.

Exemple :

```
cd [destination] ; ll
```

Les variables d'environnement

- a. **Variable d'Environnement** : Une variable d'environnement est un nom associé à une valeur, qui peut être utilisée par les programmes en cours d'exécution dans le shell Bash.

```
NOM_UTILISATEUR="john_doe"
```

- b. **Variables Système** : Il existe des variables prédéfinies qui sont utilisées par le système et le shell lui-même. « PATH » spécifie les répertoires où le shell recherche les commandes et « HOME » montre le répertoire personnel de l'utilisateur.

- c. **Affectation de valeurs** : on peut affecter une valeur à une variable avec l'opérateur égal (« = »).

```
MA_VARIABLE='valeur de la variable'
```

- d. Affichage de valeurs : on peut afficher la valeur d'une variable avec echo.

```
echo $MA_VARIABLE
```

- e. **Variables d'environnement** : ces variables sont utilisées pour configurer le comportement global du shell et des programmes.

PS1 définir le prompt du shell.

PS2 définit le prompt de continuation (une commande pour plusieurs lignes).

SHELL indique le chemin vers le shell utilisé.

- f. Variables spéciales :

- \$0 : nom de la commande ou du script en cours d'exécution.
- \$1, \$2, ... sont les paramètres positionnels passés au script ou à la commande.
- \$# : nombre total de paramètres passés.
- \$? : code de retour de la dernière commande.

- g. Exportations de variables : pour rendre une variable d'environnement accessible aux processus enfants, on utilise export

```
Export MON_CHIFFRE=42
```

- h. Variables d'environnement globales : ces variables sont disponibles pour tous les utilisateurs du système.
 - /etc/profile : Fichier de configuration pour toutes les variables globales.
 - /etc/environment : fichier pour les variables d'environnement globales.
- i. Variables d'environnement utilisateur : ces variables d'environnement sont spécifiques à un utilisateur.
 - ~/.bashrc : fichier de configuration pour les variables d'utilisateur.
 - ~/.bash_profile : fichier de configuration exécuté à la connexion.

Il existe aussi les opérations comme pour les langages de programmation (python, java, etc.). + - / *, % (modulo), ==, != (égalité), > < <= >= (comparaison), && (et), || (ou), \ (caractère d'échappement).

Les structures de contrôles

▪ Conditionnelles (if, elif, else)

if [condition]; then

instructions si la condition est vraie

elif [autre_condition]; then

instructions si l'autre condition est vraie

else

instructions si aucune des conditions précédentes n'est vraie

fi

→ Dans les conditions, vous pouvez utiliser divers opérateurs de test, tels que -eq (égal), -ne (non égal), -lt (inférieur), -le (inférieur ou égal), -gt (supérieur), -ge (supérieur ou égal), == (égal pour les chaînes de caractères), != (non égal pour les chaînes), etc.

→ Les doubles crochets offrent une syntaxe plus souple et puissante surtout pour les chaînes de caractères et les opérations logiques [[]].

▪ Boucle (for)

for variable in liste; do

instructions à exécuter pour chaque élément de la liste

done

- **Boucle (*while*)**

```
while [ condition ]; do  
    # instructions à exécuter tant que la condition est vraie  
done
```

- **Boucle (*until*)**

```
until [ condition ]; do  
    # instructions à exécuter jusqu'à ce que la condition devienne vraie  
done
```

- **Switch (*case*)**

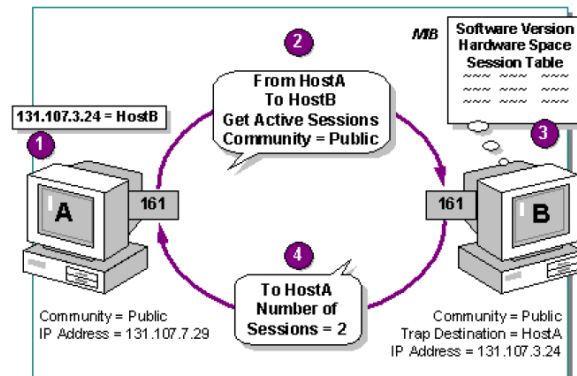
```
case $variable in  
    motif1)  
        # instructions pour motif1  
        ;;  
    motif2)  
        # instructions pour motif2  
        ;;  
    *)  
        # instructions par défaut si aucun motif ne correspond  
        ;;  
esac
```

Chapitre 16. Administration de réseaux

1. SNMP (Simple Network Management Protocol)

SNMP (Simple Network Management Protocol) est un protocole de gestion de réseau standard qui permet aux administrateurs de surveiller et de gérer les équipements réseau, tels que les routeurs, les commutateurs, les serveurs, et d'autres dispositifs. Il est largement utilisé dans les réseaux informatiques pour collecter des informations sur l'état et les performances des équipements réseau.

Principe de fonctionnement SNMP



Principe de fonctionnement de SNMP :

1. Architecture Client-Serveur :

SNMP suit un modèle client-serveur. Les dispositifs réseau (serveurs) exposent des informations gérées via des agents SNMP, et les gestionnaires SNMP (clients) recueillent ces informations pour la surveillance et la gestion du réseau.

2. Entités clés :

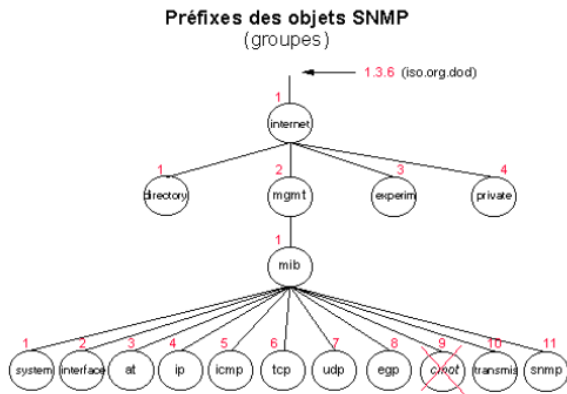
Agent SNMP : C'est un logiciel qui fonctionne sur chaque équipement réseau et collecte des informations locales. Il répond aux requêtes SNMP des gestionnaires et peut également envoyer des notifications aux gestionnaires en cas d'événements importants.

Gestionnaire SNMP : C'est une application qui collecte et analyse les informations fournies par les agents SNMP. Les gestionnaires peuvent également envoyer des commandes aux agents pour configurer les paramètres du réseau.

3. MIB (Management Information Base) :

La MIB est une base de données hiérarchique qui définit les informations que les agents SNMP peuvent fournir. Elle définit un ensemble d'objets et leurs attributs qui représentent des aspects spécifiques d'un équipement réseau ou d'un système. Chaque objet dans la MIB est identifié par un objet (**OID - Object Identifier**).

- C'est une série de chiffres qui identifie de manière unique chaque objet dans la MIB.
- Par exemple, l'OID 1.3.6.1.2.1.2.2.1.10 peut représenter le nombre d'octets reçus sur une interface réseau.



4. Protocole SNMP :

SNMP utilise les opérations GET, GETNEXT, et SET pour collecter ou configurer des informations. Les messages SNMP sont transportés sur l'UDP (User Datagram Protocol).

5. Versions de SNMP :

- SNMPv1 : La première version, basique et largement utilisée.
- SNMPv2 : Améliorations, mais plusieurs versions avec des fonctionnalités différentes (SNMPv2c, SNMPv2u, SNMPv2*).
- SNMPv3 : La version la plus sécurisée, avec des fonctionnalités avancées de sécurité et d'authentification.

6. Exemple d'utilisation :

- Un gestionnaire SNMP peut envoyer une requête GET à un agent pour obtenir des informations sur le trafic réseau d'un commutateur.
- Un agent SNMP sur un routeur peut envoyer une notification au gestionnaire lorsqu'une interface réseau est hors service.

SNMP joue un rôle crucial dans la surveillance proactive des réseaux, permettant aux administrateurs de réagir rapidement aux problèmes et d'optimiser les performances du réseau.

7. Traps (alerte, notification) :

Les traps, également appelées notifications, sont utilisées pour informer de manière proactive un gestionnaire SNMP des événements importants sur un dispositif. L'agent SNMP peut envoyer une trap au gestionnaire pour signaler des situations telles que des pannes, des erreurs critiques, ou d'autres événements pertinents. Les traps sont envoyées de manière asynchrone, sans qu'elles soient explicitement sollicitées par le gestionnaire.

8. Polling (Interrogation) :

Le polling, ou interrogation, dans le contexte de la gestion de réseau avec SNMP (Simple Network Management Protocol), est une méthode par laquelle un gestionnaire SNMP demande des

informations à un agent SNMP. Cela se fait généralement à intervalles réguliers. Voici comment cela fonctionne :

➤ **Intervalle de Polling :**

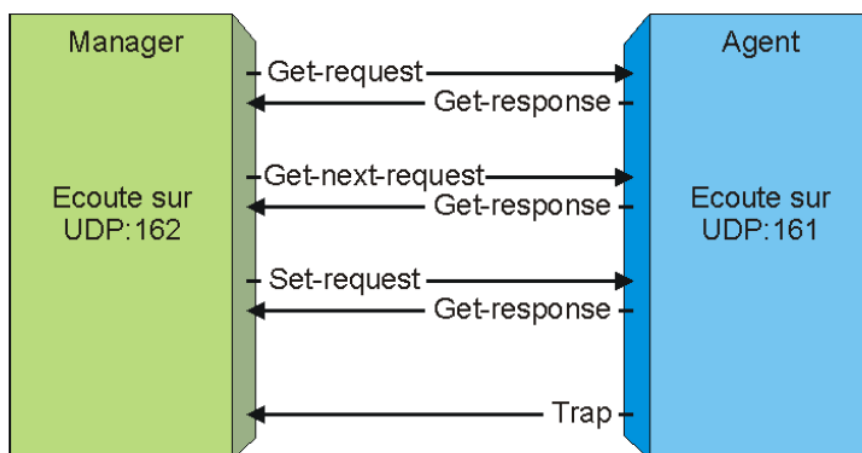
Le gestionnaire SNMP émet périodiquement des requêtes vers les agents SNMP pour obtenir des informations sur l'état et les performances du réseau. Cet intervalle peut être configuré en fonction des besoins de surveillance, par exemple, toutes les 5 minutes.

➤ **Opération de Polling :**

Lorsque le gestionnaire veut obtenir des données spécifiques d'un agent, il envoie une requête GET SNMP avec l'OID (Object Identifier) correspondant à l'information recherchée. Par exemple, pour obtenir l'utilisation du processeur sur un équipement réseau, le gestionnaire envoie une requête GET avec l'OID associé à cette information.

➤ **Réponse de l'Agent :**

L'agent SNMP, qui est installé sur les équipements réseau, répond à la requête du gestionnaire en fournissant la valeur actuelle de l'objet demandé. Cette réponse est ensuite traitée par le gestionnaire, qui peut analyser, afficher ou stocker ces données.



Chapitre 17. Introduction à la sécurité

1. Les types de menaces

▪ **Virus**

C'est un programme caché dans le corps d'un fichier en apparence anodin. Lorsque ce fichier est ouvert/exécuté, le virus exécute automatiquement les actions que son auteur a programmées.

- ***Vers (Worms)***

Virus qui se propagent généralement à travers le réseau. Ils s'autoreproduisent d'ordinateur à ordinateur.

- ***Cheval de Troie (trojan)***

Programme d'apparence légitime (souvent un petit jeu et/ou utilitaire) qui comporte une routine nuisible exécutée sans l'autorisation de l'utilisateur.

- ***Portes dérobées (backdoors)***

Programmes installés dans l'ordinateur à l'insu de l'utilisateur et qui donnent accès au contrôle de logiciels ou de l'ordinateur par des pirates.

- ***Logiciels-espions (spyware)***

Programme espion collectant, sans son autorisation, des données personnelles d'un utilisateur pour les envoyer à un tiers. (Programme souvent inclus dans un autre programme – souvent un gratuitel / partagiciel / pilote de périphérique).

- ***Adwares (publiciels)***

Logiciel gratuit dont le créateur finance ses activités en affichant de la publicité lors de l'utilisation du logiciel. Il est malveillant si l'utilisateur n'est pas averti qu'il recevra de la publicité après avoir installé le logiciel.

- ***Keylogger***

Logiciel espion capable d'enregistrer tout ce qui est tapé au clavier et qui le renvoie ensuite à un réseau de pirates.

- ***Les exploits***

Un exploit est un programme qui exploite une faille de sécurité informatique dans un système d'exploitation ou dans un logiciel afin de prendre le contrôle d'un ordinateur ou installer des malwares.

- ***Les wabbits***

Ils n'infectent ni les programmes, ni les documents. Ils interviennent par exemple dans la saisie semi-automatique des navigateurs, en incorporant des termes censés amener l'internaute sur des sites payants.

- ***Les rootkits***

Ce sont des ensembles de programmes chargés de dissimuler l'activité nuisible d'un malware.

- **Composeurs (dialers)**

Un composeur branche l'ordinateur à un numéro de téléphone dont les frais d'utilisation sont très élevés.

- **Rogues/scareware**

Faux anti-spywares sont seulement créés pour faire de l'argent. Son principe est de convaincre un utilisateur que son ordinateur contient un logiciel malveillant, puis lui suggérer de télécharger un logiciel (payant) pour l'éliminer. Le virus est le plus souvent fictif et le logiciel téléchargé peut être inutile et/ou malveillant.

- **Hoax**

Un hoax est une information fausse, périmée ou invérifiable propagée spontanément par les internautes.

- **Spam**

Messages intempestifs envoyés à une personne ou à un groupe de personnes lors d'une opération de spamming.

- **Phishing**

Courrier électronique semblant provenir d'une entreprise de confiance, typiquement une banque ou un site de commerce électronique alors qu'il provient d'escrocs usurpant l'identité d'une entreprise.

2. Les sinistres

- **Usure des appareils provoquant des pannes**

Alimentation, disque dur, ...

- **Catastrophes naturelles**

Incendie, inondation, ...

- **Erreurs humaines**

Effacer un fichier, un mail, renverser du liquide sur le matériel, ...

- **Vol ou pertes**

Clés USB, portables oubliés dans le train, ...

3. Principales sources de menaces

1. Les utilisateurs du système.
2. Les personnes malveillantes.
3. Les programmes malveillants (malware).
4. Les sinistres.

4. DRP (Disaster Recovery Plan)

L'objectif du DRP est de minimiser les interruptions d'activités en cas de sinistre, qu'il s'agisse de catastrophes naturelles, de défaillances matérielles, d'attaques informatiques, etc.

▪ **Contenu du DRP :**

- Identification des processus et des systèmes critiques.
- Planification des procédures de sauvegarde et de restauration des données.
- Mise en place de stratégies de continuité pour les opérations essentielles.
- Assignation des responsabilités au sein de l'organisation.
- Coordination avec les parties prenantes internes et externes.

▪ **Étapes typiques du DRP :**

- **Analyse des Risques :** Identifier les menaces potentielles et évaluer leur impact sur les activités.
- **Développement du Plan :** Concevoir des procédures détaillées pour la reprise des activités.
- **Mise en Œuvre :** Mettre en place les mesures préventives et les solutions de reprise.
- **Test et Exercices :** Tester régulièrement la validité du plan par le biais d'exercices de simulation.
- **Maintenance et Mise à Jour :** Adapter le plan en fonction des évolutions de l'entreprise et des technologies.

▪ **Importance du DRP :**

- La perte de données critiques ou les temps d'arrêt prolongés peuvent entraîner des conséquences financières et opérationnelles graves.
- Un DRP bien conçu contribue à atténuer ces risques et à assurer une reprise rapide des activités.

5. Que devons-nous sécuriser ?

- Les accès (contrôles des accès à une pièce, des connexions, ...).
- Les données (accès, intégrité, ...).
- Les services (accès, intégrité, ...).

6. Critères de sécurisation

- Disponibilité : accessibilité des données aux personnes autorisées à un moment t.
- Intégrité : garantir que chaque fichier est exact et complet.
- Confidentialité : Seules les personnes autorisées ont accès aux éléments.
- Traçabilité : toutes les tentatives d'accès à des systèmes sont tracés et que c'est enregistré et accessible quelque part.

7. Comment sécuriser ?

C'est un travail complexe et difficile. Il est impossible de sécuriser à 100%. Ce travail peut engendrer des problèmes de coût, de confort d'utilisation et de motivation. Il faut pouvoir mettre en place une politique de sécurité, mettre en place des contremesures et agir sur les faiblesses (mises à jour, désactiver sur l'inutile, ...).

Il faut utiliser des normes, des méthodes et des référentiels de bonnes pratiques en matière de sécurité ou, dans le cas échéant, faire appel à des professionnels dans ce domaine.

Même si la sécurité logicielle est très importante, il faut aussi protéger l'accès au matériel pour empêcher le vol (local sécurisé), éviter les maladroites (se prendre les pieds dans un câble, renverser du liquide – protéger les câbles et ne pas boire à côté des machines) et protéger le matériel contre les coupures d'alimentation (ondulateur, disjoncteur, groupe électrogène, paratonnerre, ...).

Chapitre 19. Les Firewalls

1. Principe général d'un pare-feu

Un pare-feu est un dispositif logiciel ou matériel qui filtre le flux de données sur un réseau. Ce filtrage permet de définir quels sont les types de communications autorisés ou interdits.

Le filtrage

Le pare-feu permet d'appliquer une politique d'accès aux ressources réseau en filtrant les paquets passant par lui. Voici certains des critères de ce filtrage :

- L'origine ou la destination des paquets (adresse IP, ports TCP ou UDP, ...).
- Les options contenues dans les en-têtes des paquets (fragmentation, validité, ...).
- Les données elles-mêmes (taille, ...).
- ...

Types de pare-feu

Le terme Firewall est général et recouvre des architectures et techniques bien différentes :

- Un routeur filtrant.
- Un routeur filtrant + serveur proxy caché.
- Un système Firewall logiciel ou matériel.
- Un système de filtrage de paquets + filtrage applicatif.
- Une architecture hybride avec routeurs filtrants, firewalls dédiés et proxy.
- ...

▪ **Pare-feu – filtre de paquet**

Stateless Firewall (sans états)

Le pare-feu analyse chaque paquet indépendamment des autres et le compare à une liste de règles.

Statefull firewall (avec états)

Le pare-feu permet de faire la distinction des paquets associés à une connexion des autres paquets et conserve la trace de ces connexions. Il offre plus de sécurité car il est possible de rejeter tous les paquets n'appartenant pas à une connexion.

▪ **Pare-feu – applicatif**

Ce type de pare-feu vérifie la conformité du paquet à un protocole donné (gourmand en temps de calcul et seul un nombre limité d'application est supporté).

▪ **Pare-feu – identifiant**

Réalise l'identification des connexions ce qui permet de définir des règles de filtrage par l'utilisateur, ...

▪ **Pare-feu – personnel**

Nom donné au pare-feu à états destinés à ne protéger qu'une machine, généralement une station de travail.

▪ **Pare-feu – distribué**

Il se compose de plusieurs choses :

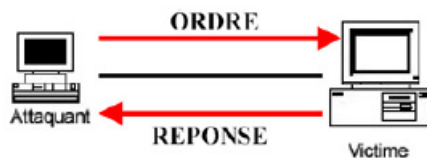
- D'un centre de partage d'administration (hébergé sur un serveur).
- D'un ensemble de pare-feu personnels installés sur chaque poste à protéger.

Sur chaque pare-feu, il peut y avoir quelques fonctionnalités en plus comme :

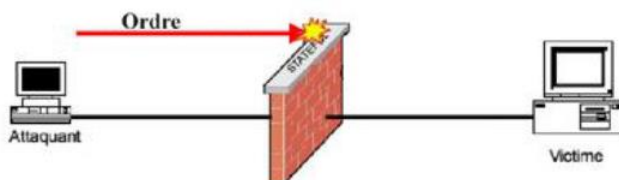
- Un module anti-virus.
- Un module anti-spam.
- Un module anti-malware/spyware.
- Une sonde de détection d'intrusion/
- Un module VPN.
- ...

Blocage des attaques par FW

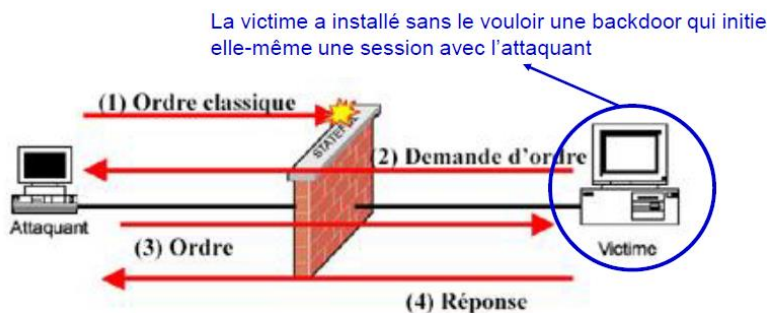
▪ Principe d'une attaque



▪ Principe de la protection par FW



Cependant le blocage des seuls flux entrants est insuffisant



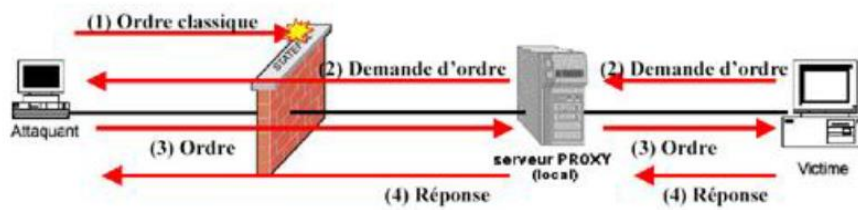
2. Les proxys

Principes généraux d'un proxy

Le proxy intercepte la demande de connexion du client puis établit une connexion, en lieu et place de l'utilisateur, avec le serveur demandé. Toutes les trames reçues peuvent être analysées pour en vérifier la cohérence. Il permet de filtrer au niveau des couches application du modèle OSI.

Protection par proxy

Un proxy joue le rôle d'un tampon entre un client et un serveur. Il héberge des applications qui filtrent ou traitent les messages avant de les transmettre à leur tour.



Il est donc possible de contrôler le flux sortant mais ce

contrôle n'est pas efficace à 100% (backdoor).

Avantages d'un proxy

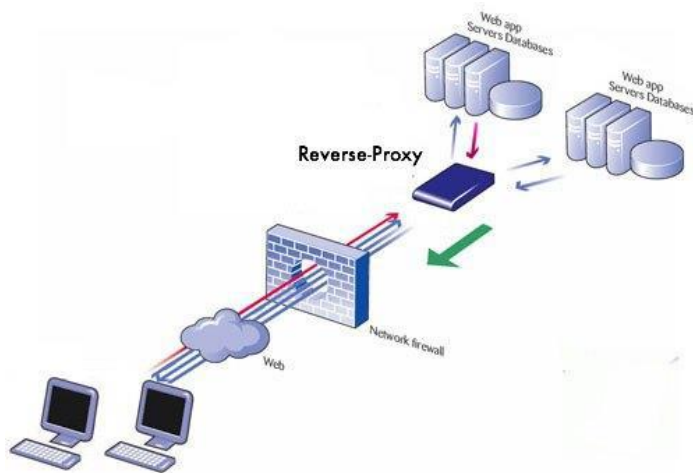
- **Contrôle et sécurité :** il donne aux administrateurs le contrôle sur les applications et protocoles spécifiques. Il est possible de filtrer sur base de liste clé, application, ...
- **Mémoire cache :** un proxy peut mettre en cache des données, permettant de réduire la consommation de bande passante.
- **Journalisation des requêtes (logging) :** les services proxy peuvent être journalisés permettant un contrôle plus strict de l'utilisation des ressources du réseau.
- **Anonymat :** toutes les requêtes sont réalisées avec l'IP du proxy.
- **Facilité de mise en place :** il n'est pas nécessaire de configurer les clients dans le cas d'un proxy transparent.

Inconvénients d'un proxy

- **Limitations :** Les serveurs proxy sont souvent particuliers à des applications (http, telnet, ...) ou limités à des protocoles (la plupart fonctionnent seulement avec des services connectés à TCP).
- **Goulot d'étranglement :** Toutes les requêtes et transmissions sont aiguillées vers le proxy plutôt que directement vers des connexions distantes.

Reverse Proxy

C'est un type de serveur proxy implémenté du côté des serveurs internet. L'utilisateur du web passe par son intermédiaire pour accéder aux applications de serveurs internes. Habituellement, c'est utilisé pour protéger des serveurs web des attaques provenant de l'extérieur.



3. Procédure pour désactiver firewalld et activer iptables

```
#systemctl stop firewalld
#systemctl disable firewalld
#yum remove firewalld
#yum install iptables
#systemctl enable iptables.service
#systemctl enable ip6tables.service
```

4. Tables, chaînes, règles, politiques et cibles

Les tables

Les tables regroupent, en fonction de leur rôle respectif (filtrer, traduire, ...) les différentes règles de traitement des paquets. Chaque table est constituée d'une ou de plusieurs chaînes.

Les chaînes

Une chaîne est constituée d'une suite de règles. Exemples de plusieurs chaînes :

- Une chaîne qui correspond aux règles de filtrage des paquets en entrée.
- Une autre pour les paquets en sortie.
- ...

Remarque :

Avec iptables, le nom des chaînes s'écrit en majuscule.

Les chaînes utilisateurs sont de nouvelles chaînes qu'il est possible de créer. Seules les chaînes prédéfinies sont utilisées donc les chaînes utilisateurs doivent être appelés à travers les règles des chaînes prédéfinies. Les chaînes utilisateurs peuvent être utilisée à plusieurs endroits.

▪ **Exemple de chaînes prédéfinies**

- INPUT : chaîne par laquelle passent les paquets entrants.
- OUTPUT : chaîne par laquelle passent les paquets sortants.
- FORWARD : chaîne utilisée par le paquet qui ne fait que transiter par le pare-feu.

Les règles

Chaque règle doit :

- Déterminer les paquets sur lesquels la règle doit s'appliquer (partie correspondante).
- Préciser le sort des paquets (partie cible) (option -j).
- Si la règle ne s'applique pas au paquet, on teste la règle suivante de la chaîne.

Les politiques

La politique d'une chaîne (option -P) détermine le sort des paquets qui atteignent le bout de la chaîne sans avoir été affecté d'une cible. **!/ \ IMPORTANT !/ ** tout ce qui n'est pas explicitement autorisé doit être **interdit** !

Les cibles

Elles spécifient soit :

- La politique d'une chaîne.
- L'action à appliquer à un paquet correspondant à une règle.

L'action (cible) est toujours écrite en majuscule.

▪ **Exemples de cibles**

- ACCEPT : autoriser le paquet à passer à l'étape suivante de traitement.
- DROP : abandon du paquet (arrêt de son traitement).
- REJECT : le paquet est rejeté et une notification de rejet est envoyée.
- LOG : permet de loguer le paquet.
- SNAT : utilisée pour changer l'adresse source.

5. Procédure pour mettre en place un firewall (iptables)

Attention, l'ordre est très important, si votre première règle est de tout refuser, les autres règles ne serviront jamais ! Les règles les plus utilisées doivent être placées en premières.

Pour voir tous les ports de tous les services → /etc/services.

▪ **Exemple de fichier iptables créé dans /etc/iptables.sh**

```
#!/bin/bash
```

```
# Efface toutes les règles existantes
```

```
iptables -F
```

```
# Bloquer tout le trafic entrant sauf sur le port 22 (SSH)
```

```
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

```
iptables -A INPUT -p udp --dport 22 -j ACCEPT
```

```
# Bloquer tout le trafic sortant sauf sur le port 22 (SSH)
```

```
iptables -A OUTPUT -p tcp --sport 22 -j ACCEPT
```

```
iptables -A OUTPUT -p udp --sport 22 -j ACCEPT
```

```
# Bloquer tout le trafic qui n'est pas autorisé
```

```
iptables -P INPUT DROP
```

```
iptables -P FORWARD DROP
```

```
iptables -P OUTPUT DROP
```

▪ **Rendre le fichier exécutable**

```
chmod +x /etc/iptables.sh
```

▪ **Créer le fichier /etc/systemd/system/[nom].service**

```
[Unit]
```

```
Description=iptables rules
```

```
[Service]
```

```
ExecStart=bash -x /etc/iptables.sh
```

[Install]

WantedBy=default.target

Si deux types d'utilisateurs, alors deux fichiers .service

- **Recharger les démons**

systemctl daemon-reload

- **Lancer et activer le service créé dans /etc/systemd/system/[nom].service**

systemctl start [nom].service

systemctl enable [nom].service

```
[root@localhost etc]# systemctl status iptables.service
o iptables.service - iptables rules
   Loaded: loaded (/etc/systemd/system/iptables.service; enabled; preset: disabled)
   Active: inactive (dead) since Sun 2024-01-07 20:45:16 EST; 5s ago
     Duration: 98ms
    Process: 1850 ExecStart=bash -x /etc/iptables.sh (code=exited, status=0/SUCCESS)
   Main PID: 1850 (code=exited, status=0/SUCCESS)
      CPU: 13ms

jan 07 20:45:16 localhost.localdomain systemd[1]: Started iptables rules.
jan 07 20:45:16 localhost.localdomain bash[1850]: + iptables -F
jan 07 20:45:16 localhost.localdomain bash[1850]: + iptables -A INPUT -p tcp --dport 22 -j ACCEPT
jan 07 20:45:16 localhost.localdomain bash[1850]: + iptables -A INPUT -p udp --dport 22 -j ACCEPT
jan 07 20:45:16 localhost.localdomain bash[1850]: + iptables -A OUTPUT -p tcp --sport 22 -j ACCEPT
jan 07 20:45:16 localhost.localdomain bash[1850]: + iptables -A OUTPUT -p udp --sport 22 -j ACCEPT
jan 07 20:45:16 localhost.localdomain bash[1850]: + iptables -P INPUT DROP
jan 07 20:45:16 localhost.localdomain bash[1850]: + iptables -P FORWARD DROP
jan 07 20:45:16 localhost.localdomain bash[1850]: + iptables -P OUTPUT DROP
jan 07 20:45:16 localhost.localdomain systemd[1]: iptables.service: Deactivated successfully.
```

➔ Accepter les pings provenant de notre réseau local

iptables -A INPUT -p icmp --icmp-type echo-request -s 192.168.1.0/24 -j ACCEPT

iptables -A OUTPUT -p icmp --icmp-type echo-reply -d 192.168.1.0/24 -j ACCEPT

6. Commandes

- Lister tous les ports ouverts d'une machine : **nmap**
- Sauvegarder une configuration : **iptables-save**
- Charger une configuration : **iptables-restore**